

UNIVERSIDAD
SERGIO ARBOLEDA

Un simulador para redes descentralizadas: cobertura y adaptabilidad de una malla B.A.T.M.A.N.

Juan Camilo Gallardo Sánchez
Jefferson Gutiérrez Tovar

Universidad Sergio Arboleda
Escuela de Ciencias Exactas e Ingeniería
Ingeniería de Ciencias de la Computación e Inteligencia Artificial
Bogotá D.C.

2026

Un simulador para redes descentralizadas: cobertura y adaptabilidad de una malla B.A.T.M.A.N.

Juan Camilo Gallardo Sánchez
Jefferson Gutiérrez Tovar

Trabajo de grado

Tutor
Joaquín Fernando Sánchez Cifuentes

Universidad Sergio Arboleda
Escuela de Ciencias Exactas e Ingeniería
Ingeniería de Ciencias de la Computación e Inteligencia Artificial
Bogotá D.C.
2026

Nota de aceptación:

Firma del presidente del jurado

Firma del jurado

Firma del jurado

UNIVERSIDAD
SERGIO ARBOLEDA

Dedicatoria

Juan Camilo:

Jefferson:

UNIVERSIDAD
SERGIO ARBOLEDA

Agradecimientos

Al profesor Joaquín Fernando Sánchez Cifuentes, tutor de este trabajo, por orientar el proyecto hacia la pregunta que le da sentido: qué necesita un simulador para redes descentralizadas, y por el simulador que sirvió de punto de partida.

Al programa de Ingeniería de Ciencias de la Computación e Inteligencia Artificial de la Universidad Sergio Arboleda, en cuyo marco se desarrolló este trabajo.

UNIVERSIDAD
SERGIO ARBOLEDA

Resumen

En una red descentralizada no existe un equipo central que vea o dirija todo el sistema: cada nodo decide con lo que alcanza a oír de sus vecinos. Para estudiar una red así antes de construirla se usan simuladores, pero las herramientas disponibles dejan vacíos. Los simuladores de propósito general evalúan una copia del protocolo escrita para la herramienta; los emuladores ejecutan el programa real, pero cada nodo consume muchos recursos y no incluyen un modelo listo de la pérdida de señal entre los pisos de un edificio, y las herramientas de redes entre pares no representan el medio de radiofrecuencia. Este trabajo implementa un simulador que atiende esos vacíos. Está escrito en Python y ejecuta en cada nodo simulado las mismas clases de enrutamiento B.A.T.M.A.N. y de detección de fallos que usa el nodo de red; lo único que reemplaza es la red, por un modelo del medio que depende de la distancia y de los pisos. El desarrollo siguió cuatro etapas: requerimientos, diseño, implementación y pruebas. El simulador envía paquetes de datos salto a salto con la tabla de rutas de cada nodo, permite elegir el tipo de paquete (UDP, ICMP o TCP), mueve los Gateway solo con información local, programa caídas de nodos, repite experimentos con semillas y se maneja desde una interfaz web local.

Las pruebas son simulaciones de una Red de Expansión de Cobertura con 7 Gateway y 6 Nodos de usuario en un edificio de tres pisos, con 10 semillas por prueba. La caída de un Gateway puente mantuvo la malla partida 90 s sin reducir la cobertura media (1.000): la cobertura, por sí sola, no muestra el daño. Dos fallos dirigidos al nodo más crítico redujeron la entrega de datos a 0.284 frente a 0.502 de dos fallos al azar (diferencia pareada de 0.218, IC 95 % [0.149, 0.287]), con una cobertura estadísticamente indistinguible. Con Gateway móviles la malla se adaptó sola: la partición por la caída del puente bajó de 90 s a 6.2 s, a costa de que la cobertura media descendiera a 0.922. Con el mismo tráfico, TCP entregó 0.924 de los paquetes, UDP 0.772 e ICMP 0.646, y TCP necesitó cerca de 2.6 veces las transmisiones de UDP. La inundación entregó más que B.A.T.M.A.N. en campo abierto, con cerca del doble de transmisiones, pero mucho menos en el edificio (0.430 contra 0.786). Las tablas de resultados se reprodujeron byte a byte en dos máquinas. El simulador no se validó contra equipos físicos ni contra el módulo batman-adv, y su modelo del medio es simple: los resultados describen esta implementación bajo este medio. Esos frentes quedan como trabajo futuro.

Palabras clave: redes descentralizadas, simulación, B.A.T.M.A.N., redes malladas, cobertura, adaptabilidad, movilidad descentralizada, reproducibilidad.

UNIVERSIDAD
SERGIO ARBOLEDA

Abstract

In a decentralized network no central device sees or controls the whole system: each node decides based on what it hears from its neighbours. Simulators are used to study such networks before building them, but the available tools leave gaps. General-purpose simulators evaluate a copy of the protocol written for the tool; emulators run the real program, but each node is costly and they offer no ready-made model of signal loss between the floors of a building; and peer-to-peer tools do not represent the radio-frequency medium. This work implements a simulator that addresses those gaps. It is written in Python and runs, in every simulated node, the same B.A.T.M.A.N. routing and failure-detection classes used by the network node; the only thing it replaces is the network, with a medium model based on distance and floors. Development followed four stages: requirements, design, implementation and testing. The simulator forwards data packets hop by hop using each node's own routing table, lets the user choose the packet type (UDP, ICMP or TCP), moves Gateways using local information only, schedules node failures, repeats experiments with seeds and is operated from a local web interface.

The tests are simulations of a Coverage Expansion Network with 7 Gateways and 6 user nodes in a three-storey building, with 10 seeds per test. Losing a bridge Gateway kept the mesh partitioned for 90 s without lowering mean coverage (1.000): coverage alone does not show the damage. Two failures targeting the most critical node reduced data delivery to 0.284 against 0.502 for two random failures (paired difference 0.218, 95 % CI [0.149, 0.287]), with statistically indistinguishable coverage. With mobile Gateways the mesh adapted by itself: the partition caused by the bridge failure fell from 90 s to 6.2 s, at the cost of mean coverage falling to 0.922. Under the same traffic, TCP delivered 0.924 of the packets, UDP 0.772 and ICMP 0.646, and TCP needed about 2.6 times the transmissions of UDP. Flooding delivered more than B.A.T.M.A.N. in open terrain, with about twice the transmissions, but far less in the building (0.430 against 0.786). Result tables were reproduced byte for byte on two machines. The simulator was not validated against physical devices or the batman-adv module, and its medium model is simple: the results describe this implementation under this medium. Those fronts remain as future work.

Keywords: decentralized networks, simulation, B.A.T.M.A.N., mesh networks, coverage, adaptability, decentralized mobility, reproducibility.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Contenido

Lista de figuras **11**

Lista de tablas **15**

Lista de símbolos **17**

Glosario **19**

1. Introducción **21**

2. Planteamiento del problema y objetivos **23**
2.1. Planteamiento del problema . . . 23
2.2. Justificación . . . 24
2.3. Objetivos . . . 24
2.3.1. Objetivo general . . . 24
2.3.2. Objetivos específicos . . . 25
2.4. Alcance y limitaciones . . . 25

3. Estado del arte **27**
3.1. Simuladores de redes de propósito general . . . 27
3.2. Herramientas orientadas a redes descentralizadas . . . 28
3.2.1. Redes ad-hoc móviles y redes malladas . . . 28
3.2.2. Redes entre pares y superposiciones descentralizadas . . . 29
3.2.3. Redes descentralizadas con consenso distribuido . . . 29
3.3. Simulación, emulación y ejecución de código real . . . 29
3.4. El protocolo B.A.T.M.A.N. en la literatura . . . 31
3.5. La credibilidad de los estudios de simulación . . . 31
3.6. Redes de expansión de cobertura . . . 32
3.7. Brecha identificada . . . 33

4. Metodología **35**
4.1. Enfoque . . . 35
4.2. Etapas del trabajo . . . 35
4.3. Escenario de las pruebas . . . 35
4.3.1. Despliegue de referencia . . . 35
4.3.2. Pruebas y corridas . . . 37
4.4. Métricas . . . 37

UNIVERSIDAD SERGIO ARBOLEDA

4.5. Análisis estadístico 38
4.6. Cómo se comprueba el simulador 39
4.7. Amenazas a la validez 39

5. Desarrollo del simulador **41**
5.1. Requerimientos 41
5.2. Diseño 41
5.2.1. Arquitectura 41
5.2.2. El protocolo en cada nodo 41
5.2.3. El modelo del medio de radiofrecuencia 45
5.2.4. Envío de paquetes de datos 46
5.2.5. Movimiento descentralizado de los Gateway 47
5.2.6. Medir lo que cree cada nodo 48
5.2.7. Otros mapas, otros movimientos y un protocolo de comparación 48
5.3. Implementación 48
5.3.1. Escenarios, lotes y formas de ejecución 49
5.3.2. Interfaz web 49
5.3.3. Costo por nodo 55
5.3.4. Diferencias con B.A.T.M.A.N. 57
5.4. Pruebas 58
5.4.1. Ejemplo de una prueba: la caída del puente 59
5.4.2. Comprobaciones automáticas del código 59

6. Resultados y discusión **61**
6.1. Funciones básicas (pruebas P1 y P2) 61
6.2. Tipos de paquete (prueba P3) 61
6.3. Fallos con Gateway fijos (prueba P4) 62
6.4. Los mismos fallos con Gateway móviles (prueba P5) 65
6.5. Modos de movimiento (pruebas P6 y P7) 66
6.6. Fallos al azar y fallos dirigidos (prueba P8) 70
6.7. B.A.T.M.A.N. e inundación (prueba P9) 71
6.8. Repetibilidad y tamaño de la red (pruebas P10 y P11) 74
6.9. Lo que las pruebas mostraron sobre el protocolo 74
6.10. Resumen de las pruebas 75
6.11. Discusión 75

7. Conclusiones y recomendaciones **79**
7.1. Conclusiones 79
7.2. Recomendaciones 80

Anexos **83**

8. Bibliografía **87**

UNIVERSIDAD
SERGIO ARBOLEDA

Lista de figuras

4.1. Despliegue de referencia, visto como un corte vertical del edificio de tres pisos. Los círculos son los Gateway (G1 a G7) y los cuadrados, los Nodos de usuario (N1 a N6). Cada línea es un enlace de radio posible entre dos nodos, y su grosor aumenta con la fiabilidad del enlace. G6 es el único enlace entre el piso 3 y el resto 36

4.2. Fiabilidad de un enlace, es decir, probabilidad de que una transmisión llegue, según la distancia entre los dos nodos. Se comparan el medio por defecto del simulador y el del despliegue de referencia, para nodos en el mismo piso y con un piso de diferencia. Más allá del alcance la fiabilidad es cero 37

5.1. Arquitectura del simulador, en capas. La capa azul (`mesh/`) es el código del nodo, que se ejecuta sin cambios; la naranja (`sim/`) es lo que el simulador modela: el tiempo, el medio y el movimiento; las grises leen el estado para analizarlo o mostrarlo. Las flechas indican qué capa usa a cuál 43

5.2. El protocolo en funcionamiento: propagación de los OGM originados por G1 (captura de la interfaz, escenario `fallo_puente`, semilla 1). Cada punto naranja es una copia del mensaje viajando por un enlace, con el número de saltos que le quedan ("ttl"). Las copias cercanas a G1 llevan ttl 6 y 5; las que ya cruzaron varios nodos, ttl 3 y 2. El círculo punteado es el alcance de radio de G1 44

5.3. Vista general de la interfaz a los 99 s de la prueba de la caída del puente (semilla 1). En el mapa, G6 aparece caído (círculo gris con una equis) y las dos zonas sombreadas son los dos grupos en que quedó partida la malla. Arriba a la derecha, el aviso rojo "Red partida (2)" y el indicador "Cubiertos 6/6": los seis usuarios siguen cubiertos. Los Gateway con borde rojo tienen alertas activas. A la derecha, la tabla de rutas de G4: en rojo, las rutas hacia el piso 3 que ya nadie refresca 50

5.4. Función "Ver como este nodo", aplicada a G7 a los 99 s de la prueba de la caída del puente. El mapa solo muestra lo que G7 sabe: las flechas continuas son sus rutas vigentes (hacia N5 y N6) y las líneas punteadas rojas son rutas que conserva en su tabla hacia nodos que ya no puede alcanzar, con los saltos, la calidad y la antigüedad de cada una. El recuadro superior resume cuántos destinos conoce y cuántos están obsoletos . . . 51

UNIVERSIDAD SERGIO ARBOLEDA

5.5. Matriz de conocimiento. Cada fila es un nodo que "sabe" y cada columna, un destino. Una casilla verde es una ruta vigente, con su número de saltos; una naranja, un par conectado por radio que todavía no tiene ruta; una roja, una ruta obsoleta. (a) Con la malla formada, 152 de los 156 pares conectados por radio ya tienen ruta (97 %). (b) Con G6 caído, las rutas entre el piso 3 y el resto aparecen en rojo: siguen en las tablas, pero ya no sirven 52

5.6. Dos paneles de la interfaz a los 99 s de la prueba de la caída del puente. (a) A la izquierda, lo que ejecutó el código del nodo (OGM y *beacons* procesados, rutas en las tablas, revisiones de fallos); a la derecha, lo que aportó el modelo: el medio intentó 8783 entregas y completó el 60.5 %. (b) Gráficas que se actualizan durante la corrida: nodos vivos, número de grupos de la malla (sube de 1 a 2 al caer el puente, zona sombreada) y calidad y longitud de las rutas 53

5.7. Panel de tráfico a los 50 s de una sesión sobre el despliegue de referencia. Arriba, el formulario para elegir origen, destino, tipo de paquete, cantidad e intervalo. En la tabla "Por tipo de paquete" se ve el resultado de enviar 20 paquetes de cada tipo de N1 a N4 y dos flujos adicionales: UDP entregó 17 de 20; ICMP, 20 de 30, con 10 ecos sin respuesta; TCP, 30 de 30, con 18 reenvíos. Abajo, los flujos en curso y los últimos paquetes con su recorrido de ida y de vuelta. En el mapa, los puntos de colores son paquetes en tránsito 54

5.8. Capas del mapa con movimiento descentralizado (escenario `base` a los 45 s). El fondo de colores es el mapa de cobertura: verde donde la señal de algún Gateway es buena, amarillo donde es débil y rojo donde no llega. Los Gateway se han separado desde su posición inicial para cubrir más área; el anillo punteado marca un nodo crítico y el círculo grande, el alcance del Gateway seleccionado (G2), cuya tabla de rutas y vecinos aparece a la derecha 55

5.9. (a) Editor de escenarios: a la izquierda, el edificio donde se ponen y se arrastran los nodos; a la derecha, la lista de nodos y los valores del edificio y del medio. (b) Laboratorio: se indica la duración, el número de semillas y los escenarios que se quieren comparar, y al terminar muestra el promedio de cada métrica 56

6.1. Cobertura a lo largo del tiempo en cada caso de la prueba P4. Cada panel es un caso: sin fallos, caída del Gateway redundante, del puente, del de borde y la cascada. La línea es el promedio de las 10 semillas y la banda de color, el rango entre la menor y la mayor. Las franjas grises marcan el tiempo en que hay algún Gateway caído. La cobertura solo baja en la caída de borde y en la cascada 63

UNIVERSIDAD SERGIO ARBOLEDA

6.2. Comparación de los cinco casos de la prueba P4 en tres métricas, una por panel: tiempo que tardan las rutas en rehacerse, número de alertas de Gateway perdido y cobertura media. Cada barra es el promedio de 10 semillas y la línea negra, la desviación estándar. “No aplica” indica que en ese caso no hubo un episodio que medir 64
6.3. Comparación de los tres modos de movimiento de los Gateway (*seguir*, *repartir* y descentralizado) en los cinco escenarios de la prueba P6. Cada panel es una métrica: cobertura media, tasa de entrega del radio, TQ medio de las rutas y tiempo con la malla partida. Cada barra es el promedio de 10 semillas. Ningún modo es el mejor en las cuatro métricas a la vez 68
6.4. Qué pasó con los paquetes enviados en la prueba P7, según el escenario y el modo de movimiento. Cada barra representa todos los paquetes enviados (suma 1) y se divide en cinco partes: entregados, perdidos por el radio en algún salto, perdidos porque un nodo no tenía ruta, perdidos por una ruta vieja o un bucle, y sin ningún camino por radio. El modo descentralizado reduce sobre todo la parte gris (sin camino), pero aumenta las pérdidas por el radio y por falta de ruta 69
6.5. Qué pasó con los paquetes enviados en las pruebas P8 (fallos al azar y dirigidos) y P9 (B.A.T.M.A.N. e inundación en campo abierto y en el edificio). Cada barra representa todos los paquetes enviados y se divide en las mismas cinco partes de la Figura 6.4. En los fallos dirigidos domina la parte gris (sin camino por radio); en el edificio, la inundación pierde la mayoría de los paquetes por el radio 72

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Lista de tablas

3.1. Herramientas de simulación y emulación revisadas. Para cada una se indica su tipo, el tipo de red para el que se usa y qué código del protocolo ejecuta: una copia escrita para la herramienta (modelo) o el programa real 30

4.1. Etapas del desarrollo. Cada fila indica el periodo, la etapa de los objetivos a la que corresponde y lo que se obtuvo 36
4.2. Métricas que produce cada corrida y su definición. Un "componente" es un grupo de nodos que pueden comunicarse entre sí por radio, directamente o a través de otros 38

5.1. Requerimientos del simulador. Para cada uno se indica de dónde sale y con qué parte del simulador se atiende 42
5.2. Valores de los parámetros del medio y del protocolo: los que el simulador trae por defecto, los del despliegue de referencia en el edificio y los del escenario de campo abierto 46
5.3. Funciones de la interfaz, figura que muestra cada una y requerimiento de la Tabla 5.1 que atiende 56
5.4. Velocidad del simulador según el tamaño de la red (una corrida, semilla 1). La última columna indica cuántos segundos de simulación se calculan en un segundo de reloj: por encima de 1, el simulador va más rápido que el tiempo real 57
5.5. Pruebas de simulación: nombre, propósito y resultado esperado de cada una 58
5.6. Métricas de una corrida de la prueba de la caída del puente (escenario `fallo_puente`, 240 s simulados, semilla 1), tal como las entrega el simulador al terminar 60

6.1. Entrega según el tipo de paquete en el despliegue de referencia 62
6.2. Respuesta de la malla a cada fallo 62
6.3. Calidad de las rutas y del medio en los experimentos de fallo 65
6.4. Respuesta de la malla a cada fallo con Gateway móviles (descentralizada) 65
6.5. Calidad de las rutas y del medio en los experimentos de fallo con Gateway móviles (descentralizada) 66
6.6. Movilidad de los Gateway: *seguir*, *repartir* y *descentralizada* 67
6.7. Plano de datos según la movilidad de los Gateway 67
6.8. Dos caídas al azar contra dos caídas dirigidas al nodo más crítico . . . 70
6.9. B.A.T.M.A.N. contra inundación en dos mapas 71

UNIVERSIDAD SERGIO ARBOLEDA

6.10. Diferencias pareadas por semilla, con intervalo de confianza del 95 % . . 73
6.11. Resumen de las pruebas: lo que se esperaba, lo que se obtuvo y si coinciden 75
6.12. Comparación del simulador con las herramientas revisadas: qué ofrece cada una. Las primeras filas son lo que le falta a este simulador; las últimas, lo que aporta 77

7.1. Comprobaciones automáticas del código, por área del simulador 85

UNIVERSIDAD
SERGIO ARBOLEDA

Lista de símbolos

Símbolos con letras latinas

Símbolo	Término	Unidad SI	Definición
a_f	Atenuación por piso de diferencia (`floor_atten`)	1	Ec. 5.2
d	Distancia entre dos nodos	m	
$\bar{D}$	Promedio de las diferencias pareadas entre dos configuraciones		Ec. 4.1
LQ	Calidad de enlace: fracción recibida de la ventana de secuencias	1	Ec. 5.1
n	Número de semillas (corridas) de una comparación	1	Ec. 4.1
$o,\ v$	Nodo que origina un OGM y vecino por el que llega		Ec. 5.1
p_0	Pérdida base del medio (`perdida_base`)	1	Ec. 5.2
R	Alcance máximo de radio (`rango_comm`)	m	Ec. 5.2
r	Fiabilidad: probabilidad de entrega de una transmisión	1	Ec. 5.2
s_D	Desviación estándar de las diferencias pareadas		Ec. 4.1
$t_{0{,}975,\,n-1}$	Valor de la distribución t de Student con $n-1$ grados de libertad	1	Ec. 4.1
TQ	Calidad de transmisión acumulada de una ruta	1	Ec. 5.1
$W_{o,v}$	Ventana de las últimas 16 secuencias de o esperadas por v; w es cada elemento (1 recibida, 0 perdida)	1	Ec. 5.1

Símbolos con letras griegas

UNIVERSIDAD
SERGIO ARBOLEDA

Símbolo	Término	Unidad SI	Definición
Δf	Diferencia de pisos entre dos nodos	1	Ec. 5.2
ϕ	Degradación de la fiabilidad con la distancia (`falloff`)	1	Ec. 5.2

Abreviaturas

Abreviatura	Término
B.A.T.M.A.N.	*Better Approach To Mobile Ad-hoc Networking*
IC	Intervalo de confianza
ICMP	*Internet Control Message Protocol*; aquí, el eco (*ping*)
MAC	*Medium Access Control*, control de acceso al medio
MANET	*Mobile Ad-hoc Network*, red ad-hoc móvil
OGM	*Originator Message*, mensaje de originador
OLSR	*Optimized Link State Routing*
P2P	*Peer-to-peer*, red entre pares
RPGM	*Reference Point Group Mobility*, movilidad en grupo por punto de referencia
TCP	*Transmission Control Protocol*, envío con confirmación
TQ	*Transmission Quality*, calidad de transmisión de una ruta
TTL	*Time To Live*, saltos restantes de un mensaje
UDP	*User Datagram Protocol*, envío sin confirmación

UNIVERSIDAD
SERGIO ARBOLEDA

Glosario

ADAPTABILIDAD: capacidad de la malla para reorganizarse por sí misma cuando un nodo se cae o se mueve: rehacer sus rutas, volver a unirse y recuperar a los usuarios sin servicio.

BEACON: mensaje periódico con el que un nodo anuncia su identidad y su posición a quienes lo oyen directamente; no se reenvía.

COBERTURA MEDIA: promedio en el tiempo de la fracción de Nodos de usuario vivos que comparten componente de radio con algún Gateway vivo.

ESTIRAMIENTO: saltos que recorrió un paquete entregado divididos por los del camino más corto por radio en el instante del envío.

GATEWAY: nodo de la malla que retransmite tráfico y extiende la cobertura; es el rol que puede moverse para reubicarse.

INUNDACIÓN: protocolo de comparación sin tablas de rutas en el que cada nodo difunde una vez cada paquete nuevo.

LOTE: archivo que define un conjunto de corridas (escenarios por semillas) y del que se obtiene un resumen estadístico.

MEDIO DE RADIOFRECUENCIA: el espacio por el que viajan las señales de radio entre los nodos; en el simulador, el modelo que decide si cada transmisión llega.

NODO CRÍTICO: punto de articulación de la malla: si cae, la red queda dividida en más componentes.

NODO DE USUARIO: nodo final que necesita cobertura de la malla.

PLANO DE DATOS: parte del simulador que lleva los paquetes de datos de los usuarios salto a salto usando la tabla de rutas de cada nodo.

RED DE EXPANSIÓN DE COBERTURA: malla de Gateway desplegada para llevar conectividad a Nodos de usuario en un área, sin coordinador central.

RUTA FANTASMA: ruta vigente en la tabla de un nodo hacia un destino con

UNIVERSIDAD
SERGIO ARBOLEDA

el que ya no existe ningún camino por radio.

RUTA VIGENTE: ruta presente en la tabla y refrescada por un OGM hace menos que el tiempo límite.

SEMILLA: valor inicial del generador de números aleatorios; con la misma semilla y configuración se repite la misma corrida.

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 1

Introducción

En una red descentralizada no hay un servidor, un controlador ni un punto desde el cual se conozca el estado completo del sistema: los nodos se comunican entre sí y cada uno reenvía los mensajes de los demás [Corson and Macker, 1999, Akyildiz et al., 2005]. Cada nodo decide con la información que le llega, que es parcial y llega con retraso. Esta característica es la que hace valiosas a estas redes, porque no dependen de una infraestructura que pueda fallar [Reina et al., 2015], y es también la que las hace difíciles de evaluar: lo que importa no está en ningún nodo en particular, sino en la diferencia entre lo que cada nodo cree y lo que realmente ocurre.

Los protocolos de red se evalúan sobre todo mediante simulación, es decir, con un programa que imita la red en un computador, porque desplegar decenas de equipos, repetir las mismas condiciones y provocar fallos a voluntad resulta costoso o imposible [Wehrle et al., 2010, Kiess and Mauve, 2007]. Sin embargo, la confianza en los estudios de simulación es una preocupación documentada desde hace dos décadas [Pawlikowski et al., 2002, Kurkowski et al., 2005], y se ha mostrado que simuladores distintos pueden dar resultados muy diferentes para un mismo protocolo [Cavin et al., 2002]. A esto se suma un problema de fondo. Un simulador de propósito general evalúa una copia del protocolo escrita para esa herramienta [Riley and Henderson, 2010, Varga and Hornig, 2008] y, como el programa conoce toda la red, resulta fácil decidir con información que ningún nodo real tendría. Un experimento así puede hablar de una red descentralizada sin respetar la condición que la define.

Este trabajo parte de un simulador de la red B.A.T.M.A.N. orientado a operaciones de búsqueda y rescate, disponible en el repositorio del tutor [Sánchez Cifuentes et al., 2026]. Al revisarlo con su orientación, el interés del proyecto pasó a una pregunta más general: qué debe tener un simulador para redes descentralizadas. La revisión encontró que la herramienta de partida, concebida para otro propósito, decidía con información global en tres puntos: los Gateway se movían mirando las posiciones de todos los nodos, los mensajes se entregaban por el camino más corto de la red real sin consultar las tablas de rutas, y no circulaban paquetes de datos. Corregir esos tres puntos se volvió parte central del trabajo, y la aplicación concreta pasó a ser una Red de Expansión de Cobertura: una malla de equipos de radio que lleva conectividad a los usuarios de un área.

El objetivo de este documento es presentar la implementación de ese simulador y las pruebas hechas con él. La herramienta ejecuta en cada nodo simulado las mismas clases de enrutamiento y de detección de fallos que usa el nodo de red, y reemplaza únicamente la red por un modelo del medio de radiofrecuencia que depende de la distancia y de los pisos. Las pruebas son simulaciones de una malla en un edificio de tres pisos, sometida

UNIVERSIDAD SERGIO ARBOLEDA

a distintos tipos de fallo, con Gateway fijos y móviles, con tres tipos de paquete y con un protocolo de inundación como comparación.

El desarrollo siguió cuatro etapas, que corresponden a los objetivos específicos: requerimientos, diseño, implementación y pruebas. Cada cifra del documento procede de un archivo de experimento guardado en el repositorio, que se puede volver a ejecutar. El trabajo se limita a un simulador que avanza por pasos de tiempo. No incluye un despliegue con equipos físicos, no representa las colisiones entre transmisiones ni el detalle del acceso al medio, y no mide el retardo de los paquetes. Estas limitaciones se tratan de manera explícita a lo largo del documento, porque condicionan lo que se puede afirmar a partir de los resultados.

El documento se organiza así. El Capítulo 2 plantea el problema, los objetivos y el alcance. El Capítulo 3 revisa las herramientas existentes y señala lo que les falta. El Capítulo 4 describe el método de trabajo y la forma de medir. El Capítulo 5 presenta el desarrollo del simulador en el orden de los objetivos: requerimientos, diseño, implementación y pruebas. El Capítulo 6 reporta y discute los resultados de las pruebas, y el Capítulo 7 responde a los objetivos y propone trabajo futuro. Los anexos indican cómo reproducir los resultados.

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 2

Planteamiento del problema y objetivos

Este capítulo formula el problema que da origen al trabajo, justifica la implementación de una herramienta propia y fija los objetivos y el alcance.

2.1. Planteamiento del problema

Quien quiera evaluar un protocolo para una red descentralizada dispone de tres tipos de herramienta, y cada uno deja por fuera algo que aquí resulta esencial (Capítulo 3). Los simuladores de propósito general, como ns-3 [Riley and Henderson, 2010] u OMNeT++ [Varga and Hornig, 2008], evalúan una copia del protocolo escrita para la herramienta, de modo que lo que se comprueba no es el programa que se instalaría en un equipo. Los emuladores, como CORE [Ahrenholz et al., 2008] o Mininet-WiFi [Fontes et al., 2015], ejecutan el programa real, pero cada nodo consume recursos del computador y no incluyen un modelo de la pérdida de señal entre los pisos de un edificio. Las herramientas para redes entre pares, como PeerSim [Montresor and Jelasity, 2009], trabajan en la capa de aplicación y no representan el medio de radiofrecuencia ni el movimiento físico de los nodos.

A estas diferencias se suma un riesgo menos visible. Un simulador conoce, por construcción, todo el sistema: las posiciones de los nodos, los enlaces y el camino más corto entre cualquier par. Si el experimento usa esa información para decidir algo que en la red real decidiría un nodo, ya no está evaluando una red descentralizada, en la que cada nodo solo dispone de información local [Corson and Macker, 1999]. El simulador del que parte este trabajo [Sánchez Cifuentes et al., 2026], concebido para otro propósito, usaba esa información global en tres lugares: el movimiento de los Gateway consultaba las posiciones de todos los nodos, los mensajes se entregaban buscando el camino en la red real, con garantía de llegada si existía alguno, y no había paquetes de datos que pusieran a prueba las tablas de rutas. Además, la calidad de transmisión (TQ), la medida con la que el protocolo elige sus rutas [Seither et al., 2011], valía siempre 1.0, porque nunca se registraban los mensajes perdidos.

Por otra parte, en una red de este tipo las preguntas de interés no se responden con una sola cifra de conectividad: qué cree cada nodo y cuánto se aleja eso de la realidad, cuánto tarda la red en notar un fallo y adaptarse, qué nodos son críticos y si la red puede reorganizarse sin que nadie la coordine. La literatura sobre la confianza en los estudios de simulación pide, además, que cada resultado se pueda repetir y que se declare qué

UNIVERSIDAD SERGIO ARBOLEDA

parte del sistema es un modelo [Pawlikowski et al., 2002, Kurkowski et al., 2005].

El problema se formula entonces con la siguiente pregunta:

> *¿Cómo implementar un simulador para redes descentralizadas que ejecute el código de enrutamiento de los nodos, represente el medio de radiofrecuencia de una edificación, tome sus decisiones solo con información local y permita medir la cobertura y la adaptabilidad de la red?*

En este documento, la *cobertura* es la fracción de usuarios que la malla alcanza, y la *adaptabilidad* es la capacidad de la malla para reorganizarse por sí misma cuando cambian las condiciones, por ejemplo cuando un nodo se cae o se mueve: rehacer sus rutas, volver a unirse y recuperar a los usuarios que quedaron sin servicio. Ambas se miden con las métricas de la Sección 4.4.

2.2. Justificación

La justificación es de tres tipos. En lo **científico**, la literatura ha señalado problemas de confianza en los estudios de simulación de redes [Kurkowski et al., 2005, Pawlikowski et al., 2002] y que los resultados dependen de la herramienta empleada [Cavin et al., 2002]. Una herramienta en la que cada corrida se define con un archivo y una semilla, y en la que se declara qué parte es programa real y qué parte es modelo, atiende directamente esos problemas.

En lo **práctico**, las mallas inalámbricas se usan para dar conectividad donde no hay infraestructura o esta ha fallado [Akyildiz et al., 2005, Reina et al., 2015]. Poder explorar con bajo costo su comportamiento antes de desplegarlas (dónde poner los equipos, qué pasa si se cae uno, cuántos hacen falta) es útil para quien las diseña, y los bancos de prueba físicos son costosos y difíciles de repetir [Kiess and Mauve, 2007].

En lo **académico**, la interfaz permite ver lo que un nodo cree, su tabla de rutas y los mensajes con los que se propaga esa información, lo que convierte a la herramienta en un recurso para enseñar cómo funcionan estas redes. Además, las reglas locales de movimiento relacionan este trabajo con la inteligencia artificial: los Gateway son agentes que, sin coordinación central y con información parcial, producen un comportamiento colectivo, como en los métodos de despliegue autónomo [Howard et al., 2002, Poduri and Sukhatme, 2004]. Esto deja un entorno de pruebas para aprender esas reglas (Sección 7.2).

2.3. Objetivos

2.3.1. Objetivo general

Implementar un simulador para redes descentralizadas que ejecute la lógica de enrutamiento B.A.T.M.A.N. de los nodos sobre un modelo del medio de radiofrecuencia

UNIVERSIDAD
SERGIO ARBOLEDA

y permita estudiar, de forma repetible y solo con decisiones locales, la cobertura y la adaptabilidad de una Red de Expansión de Cobertura ante fallos de nodos.

2.3.2. Objetivos específicos

1. **Requerimientos.** Definir los requerimientos del simulador a partir de la revisión de las herramientas de simulación y emulación existentes.

2. **Diseño.** Diseñar la arquitectura del simulador, el modelo del medio de radiofrecuencia y los mecanismos con los que los nodos deciden solo con información local.

3. **Implementación.** Implementar el simulador con escenarios configurables, envío de paquetes de datos, movimiento de los Gateway, caídas programables de nodos y una interfaz para observarlo y manejarlo.

4. **Pruebas.** Probar el simulador con simulaciones de una Red de Expansión de Cobertura que midan su cobertura y su adaptabilidad ante distintos tipos de fallo.

El Capítulo 5 sigue este mismo orden.

2.4. Alcance y limitaciones

Se incluye: un simulador en Python que avanza en pasos de 0.5 s, con uso por terminal, ventana propia e interfaz web local; escenarios en un edificio por pisos y en campo abierto; caídas y recuperaciones programables, al azar o dirigidas; tráfico de datos configurable con paquetes de tipo UDP, ICMP y TCP; tres modos de movimiento de los Gateway y tres de los usuarios; y el análisis de resultados por lotes de corridas.

No se incluye: el despliegue físico de la malla; la comparación contra el módulo batman-adv del núcleo de Linux [Open Mesh, 2026] o contra otro simulador; las colisiones, la interferencia y el detalle de la capa de acceso al medio; la medición del retardo, porque cada salto se resuelve dentro de un paso; ni la seguridad de la red.

Dos precisiones condicionan la interpretación del trabajo. Primero, el código que se ejecuta es la implementación en Python del nodo de red de este proyecto (paquete `mesh/`), no el módulo batman-adv, y difiere de este en puntos que se documentan en la Sección 5.3.4. Segundo, el modelo del medio es una función simple de la distancia y de los pisos, de modo que hereda supuestos que se saben optimistas frente a un canal real [Kotz et al., 2004]. Por ello, los resultados deben leerse como el comportamiento de esta implementación bajo este medio, y no como una predicción numérica de un despliegue físico.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 3

Estado del arte

Un protocolo de red rara vez se evalúa desplegando la red completa: el costo, la dificultad de repetir las condiciones del canal y la imposibilidad de repetir un experimento con exactitud han hecho de la simulación la herramienta más usada en el área [Wehrle et al., 2010, Kiess and Mauve, 2007]. Este capítulo revisa las herramientas de simulación de redes disponibles hoy, distingue las que permiten estudiar redes descentralizadas de las de propósito general y, a partir de esa revisión, delimita el vacío que el presente trabajo pretende cubrir.

3.1. Simuladores de redes de propósito general

La primera generación de herramientas ampliamente adoptadas por la comunidad académica se organiza alrededor del paradigma de *simulación por eventos discretos*: el estado de la red avanza saltando de un evento (transmisión, recepción, expiración de un temporizador) al siguiente, sin recorrer el tiempo de forma continua [Wehrle et al., 2010].

ns-2 y ns-3. El simulador ns-2, desarrollado en el marco del proyecto VINT, fue una herramienta central de la investigación en redes [Breslau et al., 2000]. Su sucesor, ns-3, es un simulador nuevo escrito en C++ [Riley and Henderson, 2010], cuya biblioteca de modelos incluye tecnologías inalámbricas como IEEE 802.11 y LTE [The ns-3 Project, 2026].

OMNeT++ e INET. OMNeT++ es un entorno modular de simulación por eventos discretos escrito en C++, concebido como una infraestructura genérica más que como un simulador de redes cerrado [Varga and Hornig, 2008]. La funcionalidad de red propiamente dicha la aporta el marco de trabajo INET, una biblioteca de modelos que incluye la pila de Internet, protocolos de enlace cableados e inalámbricos, modelos de movilidad y un conjunto de protocolos de enrutamiento para redes ad-hoc [INET Framework Project, 2026].

GloMoSim y QualNet. GloMoSim se propuso específicamente para la simulación paralela de redes inalámbricas de gran escala, organizando la pila de comunicaciones en capas con una API propia por capa, de modo que los modelos de una capa interactúan con los de la capa adyacente únicamente a través de esas interfaces [Zeng et al., 1998]. Su evolución comercial, QualNet, y otras herramientas propietarias como Riverbed Modeler (antes OPNET) y NetSim, ocupan el mismo espacio como productos comerciales.

El rasgo que comparten todas estas herramientas, y que resulta central para este trabajo, es que **el protocolo bajo estudio se reimplementa como un modelo**

UNIVERSIDAD SERGIO ARBOLEDA

dentro del simulador. Lo que se valida experimentalmente es, en sentido estricto, ese modelo, no la implementación que después se despliega en un dispositivo real. Existe una excepción parcial: el módulo *Direct Code Execution* de ns-3 permite ejecutar dentro del simulador programas y pilas de red de Linux sin modificarlos [Tazaki et al., 2013], a cambio de preparar ese código para el entorno del simulador.

3.2. Herramientas orientadas a redes descentralizadas

El término *red descentralizada* se emplea en la literatura con al menos tres alcances distintos, y a cada uno le corresponde una familia diferente de herramientas. Conviene separarlas antes de compararlas.

3.2.1. Redes ad-hoc móviles y redes malladas

Son redes sin infraestructura fija, en las que cada nodo actúa a la vez como terminal y como encaminador [Corson and Macker, 1999]; cuando los encaminadores forman una red troncal que da servicio a otros equipos se habla de redes malladas [Akyildiz et al., 2005]. Las herramientas disponibles se reparten entre simuladores y emuladores:

- **ns-3** incluye modelos de los protocolos de enrutamiento ad-hoc clásicos, entre ellos AODV [Perkins et al., 2003], OLSR [Clausen and Jacquet, 2003] y DSDV [Perkins and Bhagwat, 1994], sobre su modelo de canal inalámbrico [The ns-3 Project, 2026].
- **INET sobre OMNeT++** ofrece un conjunto equivalente de protocolos MANET junto con modelos de movilidad configurables [Varga and Hornig, 2008, INET Framework Project, 2026].
- **Cooja**, el simulador del sistema operativo Contiki para redes de sensores, introduce la noción de *simulación multinivel*: permite simular simultáneamente a nivel de red, de sistema operativo y de instrucción de máquina, ejecutando el *firmware* real del nodo sobre una emulación del *hardware* [Österlind et al., 2006]. TOSSIM, del ecosistema TinyOS, simula aplicaciones completas de ese sistema operativo [Levis et al., 2003].
- **EMANE** (*Extendable Mobile Ad-hoc Network Emulator*) es un marco de emulación que usa un modelo de capa física para considerar la propagación de la señal, las antenas y las fuentes de interferencia, con modelos de radio intercambiables [Adjacent Link LLC, 2026].
- **CORE** (*Common Open Research Emulator*) es un emulador en tiempo real que instancia topologías híbridas compuestas por nodos virtuales y *hardware* real,

UNIVERSIDAD SERGIO ARBOLEDA

apoyándose en la virtualización de la pila de red del sistema operativo; escala a topologías de más de un centenar de nodos virtuales sobre un único servidor [Ahrenholz et al., 2008].

- **Mininet-WiFi** extiende Mininet [Lantz et al., 2010] a redes inalámbricas definidas por *software*: cada nodo es un proceso ubicado en un espacio de nombres de red de Linux, interconectado mediante pares de interfaces virtuales, de manera que cualquier programa que corra sobre Linux corre sin modificaciones dentro de la topología emulada [Fontes et al., 2015].

3.2.2. Redes entre pares y superposiciones descentralizadas

Cuando la descentralización ocurre en la capa de aplicación y no en la capa física, el objeto de estudio deja de ser el canal inalámbrico y pasa a ser la topología lógica de la superposición (*overlay*). Las herramientas representativas son:

- **PeerSim**, un entorno muy escalable para escenarios dinámicos, en el que los protocolos se implementan específicamente para su API en Java [Montresor and Jelasity, 2009].
- **OverSim**, construido sobre OMNeT++, que combina modelos de superposiciones estructuradas y no estructuradas (Chord, Pastry, Kademlia) con la posibilidad de ejecutar aplicaciones de red reales sobre el mismo código [Baumgart et al., 2009].
- **PlanetSim** [García et al., 2005] y **PeerfactSim.KOM** [Stingl et al., 2011], marcos de simulación de superposiciones y de sistemas entre pares.

3.2.3. Redes descentralizadas con consenso distribuido

Una tercera lectura de *descentralizado* corresponde a los sistemas de registro distribuido. **SimBlock** es un simulador de red de cadena de bloques dirigido por eventos que permite modificar con facilidad el comportamiento de los nodos, de modo que puede estudiarse cómo influye ese comportamiento sobre la red en su conjunto [Aoki et al., 2019]. **BlockSim** es un marco para construir y simular modelos de eventos discretos de sistemas de cadena de bloques [Alharby and van Moorsel, 2020]. Estas herramientas modelan la propagación de bloques y el consenso, pero no la capa física ni la movilidad, por lo que quedan fuera del alcance de este trabajo.

3.3. Simulación, emulación y ejecución de código real

La revisión anterior muestra que las herramientas no se distinguen solamente por el dominio que cubren, sino por un eje más determinante: qué tan cerca está el código evaluado del código que finalmente se despliega. Pueden distinguirse tres niveles:

UNIVERSIDAD
SERGIO ARBOLEDA

- **Simulación pura.** El protocolo se reimplementa como modelo dentro del simulador. Permite redes grandes y corridas repetibles, pero deja una diferencia entre el modelo validado y la implementación desplegada [Kiess and Mauve, 2007].

- **Emulación.** El código real se ejecuta sobre un sistema operativo virtualizado o sobre espacios de nombres de red [Ahrenholz et al., 2008, Lantz et al., 2010]. La fidelidad es alta, pero cada nodo consume recursos del computador y representar el medio de radiofrecuencia requiere componentes adicionales, como EMANE [Adjacent Link LLC, 2026].

- **Enfoques híbridos o multinivel.** El código real del protocolo se ejecuta sobre un modelo del medio, como en la simulación multinivel de Cooja [Österlind et al., 2006] o en la ejecución directa de código de ns-3 [Tazaki et al., 2013].

La Tabla 3.1 resume las herramientas revisadas según estos criterios.

Cuadro 3.1: Herramientas de simulación y emulación revisadas. Para cada una se indica su tipo, el tipo de red para el que se usa y qué código del protocolo ejecuta: una copia escrita para la herramienta (modelo) o el programa real

Herramienta	**Tipo**	**Dominio principal**	**Código del protocolo**
ns-3	Simulador	Redes en general, MANET	Modelo reimplementado; código de Linux con *Direct Code Execution*
OMNeT++ / INET	Simulador	Redes en general, MANET	Modelo reimplementado
GloMoSim / QualNet	Simulador	Redes inalámbricas a gran escala	Modelo reimplementado
Cooja	Multinivel	Redes de sensores (Contiki)	Firmware real sobre *hardware* emulado
EMANE	Emulador	MANET, capa física detallada	Código real del sistema
CORE	Emulador	Topologías híbridas en tiempo real	Código real del sistema
Mininet-WiFi	Emulador	Redes inalámbricas definidas por *software*	Binarios reales de Linux
PeerSim	Simulador	Superposiciones entre pares	Modelo abstracto
OverSim	Simulador	Superposiciones estructuradas	Modelo y aplicaciones reales
SimBlock	Simulador	Redes de cadena de bloques	Modelo de propagación

UNIVERSIDAD
SERGIO ARBOLEDA

3.4. El protocolo B.A.T.M.A.N. en la literatura

B.A.T.M.A.N. (*Better Approach To Mobile Ad-hoc Networking*) se especificó como borrador de la IETF, hoy vencido y sin categoría de estándar [Neumann et al., 2008]. A diferencia de un protocolo de estado de enlace como OLSR [Clausen and Jacquet, 2003], ningún nodo construye una visión completa de la topología: cada nodo conoce únicamente cuál de sus vecinos es el mejor siguiente salto hacia cada destino. El borrador de 2008 decide contando mensajes de originador (OGM) recibidos; batman-adv usa una métrica de calidad de transmisión (TQ), derivada de la fracción de OGM propios que un vecino recibe correctamente [Seither et al., 2011].

Su implementación de referencia, *batman-adv*, es un módulo que opera en la capa de enlace [Open Mesh, 2026]. En cuanto a su evaluación, existen estudios experimentales: Murray et al. comparan OLSR, Babel [Chroboczek and Schinazi, 2021] y B.A.T.M.A.N. en una red ad-hoc multisalto, con Babel como el de mejor desempeño [Murray et al., 2010]; Seither et al. estudian batman-adv en un entorno real de oficina, con AODV como referencia [Seither et al., 2011]; y, más recientemente, Diniz et al. comparan OLSR, batman-adv y Babel para enjambres de drones con dispositivos Raspberry Pi [Diniz et al., 2025]. Estos trabajos dan la referencia más fiel, pero su costo limita la cantidad de nodos, la repetición de las corridas y la posibilidad de provocar fallos a voluntad. La distancia entre simulación y realidad tampoco es menor: Kiess y Mauve recogen que incluso la inundación básica se comportó en prototipos de forma distinta a lo que predecían las simulaciones [Kiess and Mauve, 2007].

3.5. La credibilidad de los estudios de simulación

Elegir la herramienta no es una decisión neutra. Cavin et al. ejecutaron un mismo algoritmo de inundación en tres simuladores y obtuvieron diferencias importantes en los resultados [Cavin et al., 2002]. Pawlikowski et al. alertaron sobre la falta de rigor estadístico en estudios de redes de telecomunicaciones [Pawlikowski et al., 2002], y Kurkowski et al., al revisar las memorias de 2000 a 2005 del simposio MobiHoc, encontraron deficiencias importantes y catalogaron errores comunes en los estudios de simulación de redes móviles [Kurkowski et al., 2005]. Por su parte, Kotz et al. contrastaron supuestos habituales de los modelos de radio con mediciones de un experimento de enrutamiento en exteriores, mostraron su debilidad, entre otras cosas frente a las asimetrías de enlace, y cómo hacen que la simulación difiera de los resultados experimentales [Kotz et al., 2004].

De esta línea se extraen dos exigencias que este trabajo adopta. La primera, que cada resultado sea reproducible a partir de una configuración declarada y que se reporte con su variabilidad. La segunda, que quede explícito qué parte del sistema es un modelo, y qué supuestos del modelo limitan las conclusiones.

UNIVERSIDAD
SERGIO ARBOLEDA

3.6. Redes de expansión de cobertura

Las redes ad-hoc multisalto se han estudiado de forma sistemática como respuesta a la falta de comunicaciones cuando la infraestructura fija no existe, está destruida o se encuentra saturada, por ejemplo en zonas de desastre [Reina et al., 2015]. Las propiedades que las hacen adecuadas son el despliegue rápido, la configuración mínima, la conectividad multisalto y la capacidad de repararse solas ante la pérdida de nodos [Akyildiz et al., 2005, Reina et al., 2015].

Este trabajo toma de allí su caso de aplicación, llamado aquí *Red de Expansión de Cobertura*: una malla de radios (Gateway) que lleva conectividad a nodos finales (Nodos de usuario) sobre un área, sin un coordinador central. Las preguntas de interés son si los usuarios quedan cubiertos, si la malla sigue conectada y cómo se degrada y se recupera ante un fallo. Tres líneas de trabajo son directamente relevantes:

Autodespliegue con fuerzas virtuales. Howard et al. proponen desplegar una red de sensores móviles en un entorno desconocido mediante campos potenciales, en los que cada nodo es repelido por los demás y por los obstáculos, con un método distribuido y escalable [Howard et al., 2002]. Poduri y Sukhatme maximizan la cobertura con la restricción de que cada nodo conserve un número mínimo de vecinos [Poduri and Sukhatme, 2004], lo que hace explícito el compromiso entre cubrir más área y mantener la red conectada. Zou y Chakrabarty combinan fuerzas de atracción y repulsión para mejorar la cobertura de un despliegue inicial, en redes organizadas en clústeres, de modo que no se trata de un método plenamente local [Zou and Chakrabarty, 2003]. La idea sigue vigente en redes de drones: Fang et al. la usan en el autodespliegue de grupos de drones para redes de emergencia [Fang et al., 2021], y Liu et al. construyen con ella una red conectada que cubre a usuarios en tierra [Liu et al., 2024]. La movilidad descentralizada de este trabajo (Sección 5.2.5) pertenece a esta familia.

Modelos de movilidad. Los modelos sintéticos más empleados para estudiar redes ad-hoc, entre ellos el de *random waypoint*, se revisan en [Camp et al., 2002]; para los usuarios que se desplazan juntos existe el modelo de movilidad en grupo por punto de referencia (RPGM) [Hong et al., 1999]. Ambos se incorporaron al simulador para no atar sus resultados a una sola suposición sobre el movimiento de los usuarios.

Fallos al azar y ataques dirigidos. Albert et al. mostraron que las redes libres de escala toleran muy bien la eliminación de nodos al azar y, sin embargo, son vulnerables a la eliminación de sus nodos más conectados [Albert et al., 2000]. Una malla física pequeña no es una red de ese tipo, pero la distinción motiva comparar aquí las dos formas de fallo; para identificar los nodos cuya caída parte la red, se usan los puntos de articulación del grafo, calculables en tiempo lineal con búsqueda en profundidad [Hopcroft and Tarjan, 1973]. Por último, la inundación, el protocolo más simple de difusión, es la comparación natural de un protocolo de rutas, pero se sabe que sus retransmisiones redundantes pueden saturar el medio [Ni et al., 1999].

UNIVERSIDAD
SERGIO ARBOLEDA

3.7. Brecha identificada

De la revisión anterior se desprenden cuatro observaciones que, en conjunto, delimitan el aporte de este trabajo:

1. Los simuladores de propósito general y los orientados a MANET (Sección 3.1) evalúan un modelo del protocolo, no su implementación. Las conclusiones obtenidas sobre B.A.T.M.A.N. en esas herramientas son, por tanto, conclusiones sobre el modelo. La ejecución directa de código de ns-3 [Tazaki et al., 2013] reduce esa diferencia, pero está pensada para programas de Linux y exige adaptar su compilación al simulador.

2. Los emuladores (Sección 3.2.1) sí ejecutan código real, pero al precio de un costo por nodo considerable en el anfitrión y sin un modelo de propagación tridimensional listo para usar que dé cuenta de la atenuación entre pisos de una edificación, un efecto medido en la literatura de propagación en interiores [Seidel and Rappaport, 1992] y que es precisamente la condición del escenario de interés.

3. Las herramientas de redes descentralizadas de las Secciones 3.2.2 y 3.2.3 operan en la capa de aplicación y no modelan ni el medio de radiofrecuencia ni la movilidad física de los nodos.

4. Hasta donde alcanza esta revisión, medir lo que cada nodo cree frente a lo que ocurre físicamente, o garantizar que las decisiones de los nodos no usen información global, depende de la disciplina de quien instrumenta el experimento: no forma parte del flujo de trabajo habitual de estas herramientas, y la literatura sobre la confianza en la simulación muestra que esa disciplina falla con frecuencia [Kurkowski et al., 2005].

Queda entonces un punto intermedio poco explorado: ejecutar la **implementación del protocolo**, es decir, las mismas clases que usa el nodo de red, dentro de un solo programa, y reemplazar únicamente la red por un modelo del medio de radiofrecuencia que tenga en cuenta la forma del edificio y la pérdida de señal entre pisos. Esta estrategia acerca el código evaluado al que se ejecuta en el nodo, sin virtualizar un sistema por cada uno, y es la que adopta el simulador desarrollado en este trabajo. Conviene precisar que la implementación que se ejecuta aquí es la del nodo en Python de este proyecto, no el módulo batman-adv; sus diferencias se documentan en la Sección 5.3.4.

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 4

Metodología

Este capítulo describe cómo se trabajó y cómo se mide. El propósito es que cualquier cifra del Capítulo 6 se pueda rastrear hasta un archivo de configuración.

4.1. Enfoque

El trabajo es una investigación aplicada, cuantitativa y experimental. El simulador es a la vez el producto del trabajo y el instrumento con el que se obtienen los resultados. Por eso se le exigen dos condiciones, tomadas de la literatura sobre la confianza en los estudios de simulación [Pawlikowski et al., 2002, Kurkowski et al., 2005]: que cada resultado se pueda repetir y que quede claro qué parte del sistema es programa real y qué parte es modelo. El desarrollo fue iterativo: cada avance se comprobó antes de pasar al siguiente, y cuando un cambio no debía alterar los resultados se volvieron a correr los experimentos para confirmarlo.

4.2. Etapas del trabajo

El trabajo se organizó en las cuatro etapas de los objetivos específicos (Sección 2.3.2): requerimientos, diseño, implementación y pruebas. La Tabla 4.1 muestra cómo se repartieron en el tiempo. Hubo dos cambios de rumbo que explican varias decisiones. El proyecto partió de un simulador de búsqueda y rescate [Sánchez Cifuentes et al., 2026]. En septiembre de 2026 pasó a ser una Red de Expansión de Cobertura y se retiró el mecanismo de "hallar supervivientes". Luego, con la indicación del tutor de centrar el problema en lo que necesita un simulador para redes descentralizadas, se revisó la herramienta con ese criterio y se encontraron los tres usos de información global que describe la Sección 2.1.

4.3. Escenario de las pruebas

4.3.1. Despliegue de referencia

La mayoría de las pruebas usa un mismo despliegue: 7 Gateway y 6 Nodos de usuario en un edificio de 40 m de ancho y tres pisos de 10 m, visto de lado (Figura 4.1). Se diseñó para que tuviera los tipos de nodo que importan en una malla: G1, G2, G4 y G5 forman un núcleo en los pisos 1 y 2, con varios caminos entre ellos; G6 es el único

UNIVERSIDAD SERGIO ARBOLEDA

Cuadro 4.1: Etapas del desarrollo. Cada fila indica el periodo, la etapa de los objetivos a la que corresponde y lo que se obtuvo

Periodo (2026)	**Etapa**	**Resultado**
Agosto	Requerimientos	Revisión de las herramientas existentes y del simulador de partida; lista de requerimientos
Agosto y septiembre	Diseño	Separación en capas (`mesh/`, `sim/`, `analysis/`); modelo del medio; escenarios en archivos; roles de Gateway y Nodo de usuario
Septiembre y octubre	Implementación	Ejecución sin ventana, semillas y lotes; interfaz web; caídas programables; envío de paquetes de datos; movimiento descentralizado; campo abierto e inundación
Octubre y noviembre	Pruebas	Simulaciones de fallos, de movimiento, de tipos de paquete y de comparación de protocolos; análisis y redacción

puente hacia el piso 3; G7 es el único Gateway al alcance de N6, y G3 el único al alcance de N2.

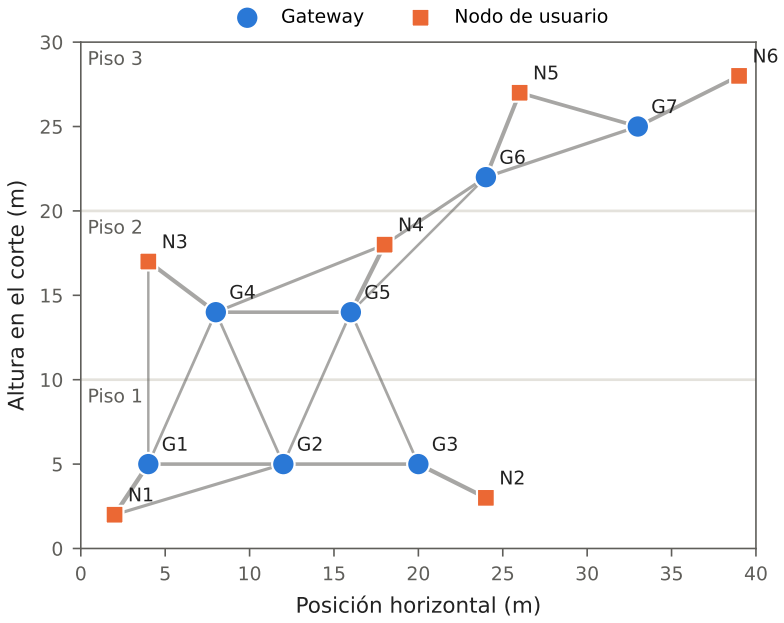


Figura 4.1: Despliegue de referencia, visto como un corte vertical del edificio de tres pisos. Los círculos son los Gateway (G1 a G7) y los cuadrados, los Nodos de usuario (N1 a N6). Cada línea es un enlace de radio posible entre dos nodos, y su grosor aumenta con la fiabilidad del enlace. G6 es el único enlace entre el piso 3 y el resto

En este despliegue el alcance de radio es de 12 m, la degradación con la distancia es $\phi = 0{,}5$ y la atenuación por piso es $a_f = 0{,}8$ (estos símbolos se definen en la

UNIVERSIDAD SERGIO ARBOLEDA

Sección 5.2.3); representan equipos de baja potencia para interiores. Se eligieron estos valores en lugar de los valores por defecto del simulador (Figura 4.2) por una razón práctica: con el medio por defecto, un despliegue con un puente y un borde en este edificio necesita enlaces tan largos que la entrega baja a 0.32, y las falsas alarmas ocultan los fallos que se quieren estudiar. Esta elección es un supuesto del experimento y no una calibración con equipos reales.

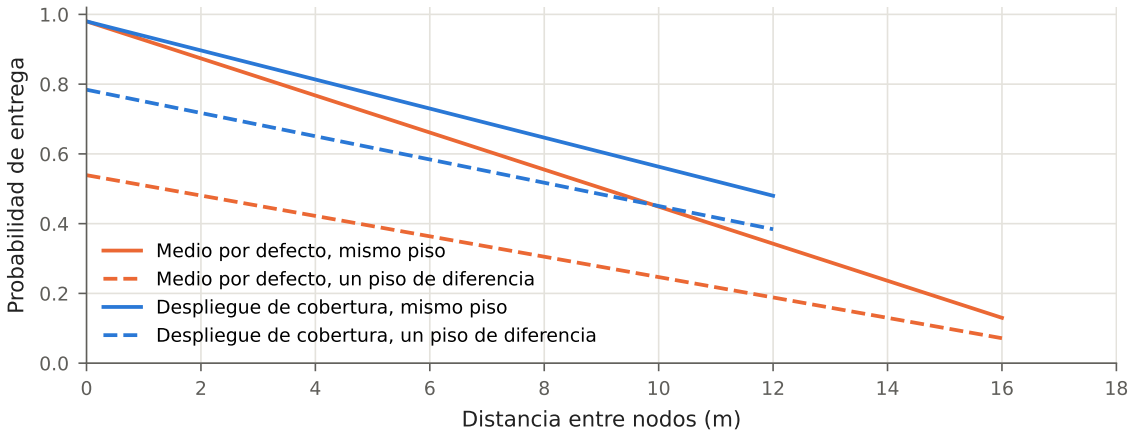


Figura 4.2: Fiabilidad de un enlace, es decir, probabilidad de que una transmisión llegue, según la distancia entre los dos nodos. Se comparan el medio por defecto del simulador y el del despliegue de referencia, para nodos en el mismo piso y con un piso de diferencia. Más allá del alcance la fiabilidad es cero

4.3.2. Pruebas y corridas

Las pruebas son simulaciones. Cada una se define en un archivo de lote del repositorio, que indica los escenarios, las semillas y la duración. El catálogo completo, con el nombre, el propósito y el resultado esperado de cada prueba, está en la Sección 5.4. En total son 45 configuraciones por 10 semillas, es decir, 450 corridas.

Además del despliegue de referencia se usan cinco escenarios heredados del repositorio base (`base`, `colapso_progresivo`, `particion`, `rescatista_perdido` y `denso`) para comparar los modos de movimiento; el nombre `rescatista_perdido` se conserva para poder rastrearlo y designa simplemente a un Gateway que se aleja del grupo. Los modos de movimiento *seguir* y *repartir* usan información global a propósito: son la referencia centralizada contra la que se mide lo que se gana o se pierde al decidir localmente.

4.4. Métricas

Cada corrida produce las métricas de la Tabla 4.2. Se separan las que describen la física (qué nodos se pueden conectar por radio) de las que describen el protocolo o lo

UNIVERSIDAD
SERGIO ARBOLEDA

que creen los nodos, porque la diferencia entre unas y otras es justamente lo que se quiere estudiar. La cobertura se mide con la cobertura media; la adaptabilidad, con el tiempo que la malla pasa partida, el tiempo que tardan las rutas en rehacerse y la entrega de datos durante y después de un fallo.

Cuadro 4.2: Métricas que produce cada corrida y su definición. Un "componente" es un grupo de nodos que pueden comunicarse entre sí por radio, directamente o a través de otros

Métrica	**Definición**
Cobertura media	Promedio en el tiempo de la fracción de Nodos de usuario vivos que están en el mismo componente que algún Gateway vivo
Malla partida (s)	Tiempo con los Gateway vivos repartidos en más de un componente. Los enlaces se cuentan entre todos los nodos vivos, de modo que un Nodo de usuario también puede unir dos grupos
Reconvergencia de rutas (s)	Tiempo desde que vuelve un Gateway o se reunifica la malla hasta que todo par de Gateway conectados por radio tiene una ruta vigente. "No aplica" si no hubo un episodio completo
Alertas	Veces en que un Gateway deja de oír a otro durante más del tiempo límite (30 s)
TQ medio y saltos medios	Promedios sobre las rutas de las tablas de todos los nodos vivos
Tasa de entrega del radio	Transmisiones entregadas sobre intentadas por el medio
Entrega de datos	Paquetes entregados sobre enviados, llevados por las tablas de rutas
Entrega con camino	Lo mismo, contando solo los paquetes cuyo destino estaba vivo y tenía algún camino por radio al enviarse. Separa el enrutamiento de la física
Estiramiento	Saltos recorridos sobre saltos del camino más corto por radio al enviar
Causas de pérdida	Fracción de los paquetes que se perdió por cada causa: sin camino por radio, sin ruta, ruta vieja, salto perdido por el radio o bucle
Transmisiones por paquete	Intentos de envío por cada paquete de datos. No incluye los mensajes de control del protocolo
Exactitud del conocimiento	Fracción de pares ordenados (a, b) de nodos vivos en que a cree lo correcto: tiene ruta vigente hacia b si y solo si el radio los conecta
Rutas fantasma y faltantes	Fracción de pares con ruta vigente pero sin conexión por radio, y con conexión por radio pero sin ruta vigente
Nodos críticos	Promedio en el tiempo del número de nodos cuya caída partiría la red

4.5. Análisis estadístico

El simulador usa números al azar, por ejemplo para decidir si una transmisión llega. La *semilla* es el valor con el que arranca el generador de esos números: con la

UNIVERSIDAD SERGIO ARBOLEDA

misma semilla y la misma configuración se repite exactamente la misma corrida. Cada configuración se corre con 10 semillas (1 a 10) y se reporta como media ± desviación estándar. Las métricas que no aplican en una corrida no entran en el promedio.

Para comparar dos configuraciones se usan diferencias pareadas por semilla [Jain, 1991]: para cada semilla se resta el resultado de las dos configuraciones, y con esas diferencias se construye el intervalo de confianza de la Ecuación (4.1):

$$\bar{D} \pm t_{0{,}975,\,n-1}\,\frac{s_D}{\sqrt{n}}, \tag{4.1}$$

donde n es el número de semillas (10); $\bar{D}$ es el promedio de las n diferencias; s_D es su desviación estándar, y $t_{0{,}975,\,n-1}$ es el valor de la distribución t de Student con $n-1$ grados de libertad que deja el 95 % de la probabilidad en el centro (2.262 para $n = 10$). Si el intervalo no contiene el cero, la diferencia se considera concluyente.

Comparar con la misma semilla reduce la variación debida al azar, pero no la elimina, porque las corridas se separan en cuanto cambia la configuración. No se aplica corrección por hacer varias comparaciones, de modo que un intervalo que excluye el cero por poco debe leerse como un indicio y no como una prueba. Con 10 semillas tampoco se comprueba que las diferencias sigan una distribución normal.

4.6. Cómo se comprueba el simulador

El simulador se comprueba de cuatro maneras:

- **Pruebas de simulación:** escenarios con un resultado esperado, que se comparan con lo que el simulador produce (Sección 5.4).
- **Cálculo a mano:** cuando una métrica se puede calcular a partir del escenario, se compara con la simulación (Sección 6.3).
- **Repetibilidad:** la misma semilla y la misma configuración deben dar la misma corrida, también entre la terminal y la interfaz web y entre computadores distintos.
- **Comprobaciones automáticas del código:** un conjunto de verificaciones que se ejecuta con una orden y avisa si un cambio rompió algo (Anexo B).

4.7. Amenazas a la validez

Lo que se mide. La cobertura se define como estar en el mismo componente de radio que algún Gateway, lo que no garantiza un enlace de buena calidad ni que exista una ruta en las tablas. Por eso se reporta junto a la entrega de datos.

Decisiones de los autores. Hay valores elegidos por los autores: los umbrales del movimiento descentralizado salieron de una búsqueda sobre cuatro de los escenarios

UNIVERSIDAD SERGIO ARBOLEDA

en los que luego se compara, y eso favorece a ese modo. El despliegue de referencia se diseñó para tener puentes y bordes, de modo que no representa a un despliegue típico.

Generalización. El modelo del medio es simple y simétrico, sin colisiones ni cambios en el tiempo, supuestos que la literatura ha mostrado optimistas [Kotz et al., 2004]. Por ello, las magnitudes absolutas no se trasladan a un despliegue físico, aunque las comparaciones relativas sí sirven para orientar hipótesis.

Conclusiones estadísticas. Diez semillas dan intervalos amplios en las métricas más variables. En algunas tablas un solo caso domina la dispersión, como se señala al discutirlas.

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 5

Desarrollo del simulador

Este capítulo presenta el desarrollo del simulador en el mismo orden de los objetivos específicos (Sección 2.3.2): primero los requerimientos, luego el diseño, después la implementación y por último las pruebas. Cada sección corresponde a un objetivo.

5.1. Requerimientos

Los requerimientos salen de dos fuentes. La primera es lo que les falta a las herramientas revisadas (Sección 3.7). La segunda son las exigencias de la literatura sobre la confianza en la simulación (Sección 3.5): que los resultados se puedan repetir y que se declare qué es modelo [Pawlikowski et al., 2002, Kurkowski et al., 2005]. La Tabla 5.1 los presenta. Los siete primeros dicen qué debe hacer el simulador; los tres últimos, cómo debe poder usarse.

5.2. Diseño

5.2.1. Arquitectura

El simulador se organiza en las capas de la Figura 5.1. La idea central del diseño es que la capa `mesh/`, que contiene el código del nodo, no sabe que está siendo simulada. La capa `sim/` crea un objeto de esas clases por cada nodo y reemplaza lo único que no puede existir dentro de un simulador, la red real, por un modelo del medio. Las capas de arriba solo leen ese estado o le envían órdenes.

El tiempo avanza en pasos de 0.5 s. En cada paso el simulador hace cinco cosas, siempre en el mismo orden: aplica los eventos programados (por ejemplo, la caída de un nodo), mueve a los nodos, hace que cada nodo procese los mensajes que recibió y emita los suyos, envía los paquetes de datos con las tablas ya actualizadas y guarda una muestra de las métricas. Todos los números al azar salen de un único generador, de modo que una semilla y una configuración determinan la corrida completa (requerimiento R6).

5.2.2. El protocolo en cada nodo

B.A.T.M.A.N. funciona con una idea sencilla: cada nodo anuncia periódicamente que existe, y los demás recuerdan por cuál vecino les llegó mejor ese anuncio [Neumann et al., 2008]. Ningún nodo conoce el mapa completo; solo sabe a qué vecino entregarle un paquete para cada destino. En el simulador cada nodo emite dos tipos de mensaje:

UNIVERSIDAD SERGIO ARBOLEDA

Cuadro 5.1: Requerimientos del simulador. Para cada uno se indica de dónde sale y con qué parte del simulador se atiende

Req.	**Requerimiento**	**De dónde sale**	**Cómo se atiende**
R1	Ejecutar el código de enrutamiento de los nodos, no una copia escrita aparte	Los simuladores generales evalúan un modelo (Sección 3.1)	Cada nodo simulado usa las clases `BatmanRouter` y `FaultManager` del nodo de red
R2	Dejar claro qué parte es programa real y qué parte es modelo	Exigencia de la literatura (Sección 3.5)	El medio es lo único que se reemplaza; un panel de la interfaz lo muestra
R3	Representar el medio de radiofrecuencia de un edificio	Los emuladores no traen un modelo por pisos (Sección 3.7)	Modelo con distancia, alcance, pérdida base y atenuación por piso
R4	Decidir solo con información local	Riesgo de usar la vista global del simulador (Sección 2.1)	Paquetes llevados por las tablas de cada nodo; movimiento basado en lo que cada Gateway oye
R5	Medir lo que cree cada nodo frente a lo que ocurre	No es parte del flujo habitual de las herramientas (Sección 3.7)	Exactitud del conocimiento, rutas fantasma y faltantes, nodos críticos
R6	Poder repetir cada resultado	Exigencia de la literatura (Sección 3.5)	Escenario en archivo, semilla, lote guardado y un solo generador de números al azar
R7	Costo bajo por nodo	Los emuladores consumen recursos por cada nodo (Sección 3.2.1)	Un solo programa, sin máquinas virtuales; su alcance real se mide en la Sección 5.3.3
R8	Provocar fallos a voluntad	Estudiar la adaptabilidad exige fallos repetibles	Caídas y recuperaciones programadas, al azar o dirigidas al nodo más crítico
R9	Enviar tráfico de datos y elegir su tipo	El simulador de partida no tenía tráfico (Sección 2.1)	Flujos de paquetes UDP, ICMP o TCP, con cantidad e intervalo configurables
R10	Poder observar y manejar la simulación	Uso para diseño y enseñanza (Sección 2.2)	Interfaz web local, ventana propia y uso por terminal

UNIVERSIDAD
SERGIO ARBOLEDA

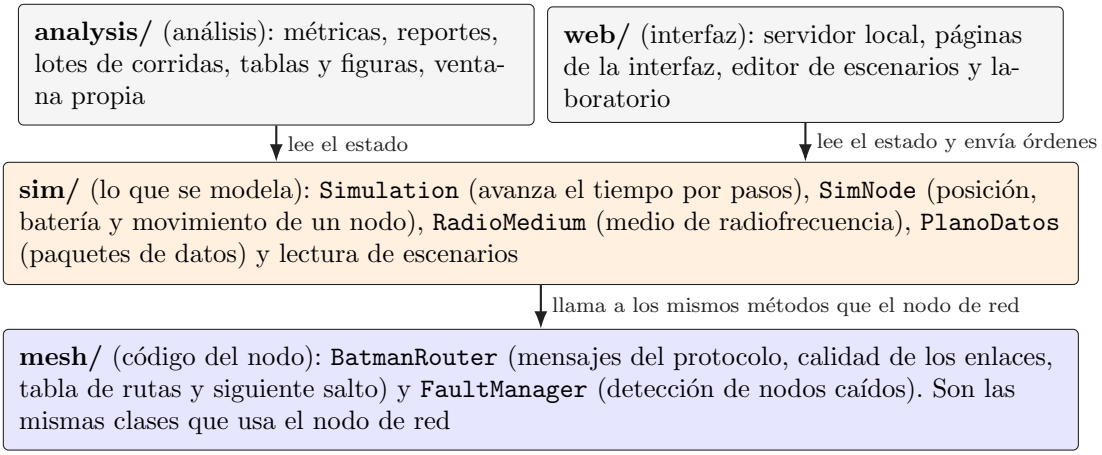


Figura 5.1: Arquitectura del simulador, en capas. La capa azul (`mesh/`) es el código del nodo, que se ejecuta sin cambios; la naranja (`sim/`) es lo que el simulador modela: el tiempo, el medio y el movimiento; las grises leen el estado para analizarlo o mostrarlo. Las flechas indican qué capa usa a cuál

- El ***beacon***, cada 2 s por defecto, anuncia la identidad y la posición del nodo a quienes lo oyen directamente. No se reenvía.
- El **OGM** (mensaje de originador), cada 4 s por defecto, es el anuncio del protocolo. Lleva un número de secuencia, un contador de saltos restantes (TTL) que empieza en 6 y una calidad (TQ) que empieza en 1.0. Quien lo recibe por primera vez lo reenvía con el TTL reducido en uno. Todos los nodos vivos reenvían OGM, también los Nodos de usuario.

La Figura 5.2 muestra este mecanismo en el simulador: los OGM que originó G1 se alejan de él salto a salto, y el TTL que lleva cada copia baja a medida que se aleja.

Calidad de enlace y TQ. Los enlaces de radio pierden mensajes. Para saber qué tan bueno es un vecino, `BatmanRouter` lleva la cuenta de cuántos de los últimos 16 OGM de cada origen le llegaron por ese vecino, como en batman-adv [Seither et al., 2011]. Con esa cuenta calcula la calidad del enlace y la calidad de la ruta, según la Ecuación (5.1):

$$LQ_{o,v} = \frac{1}{|W_{o,v}|} \sum_{w \in W_{o,v}} w, \qquad TQ_{o,v} = TQ_{\text{OGM}} \cdot LQ_{o,v}, \tag{5.1}$$

donde:

- o es el nodo que originó el OGM y v es el vecino por el que llegó;
- $W_{o,v}$ es la ventana de las últimas 16 secuencias de o esperadas por el vecino v, y $|W_{o,v}|$ es su tamaño;

UNIVERSIDAD SERGIO ARBOLEDA

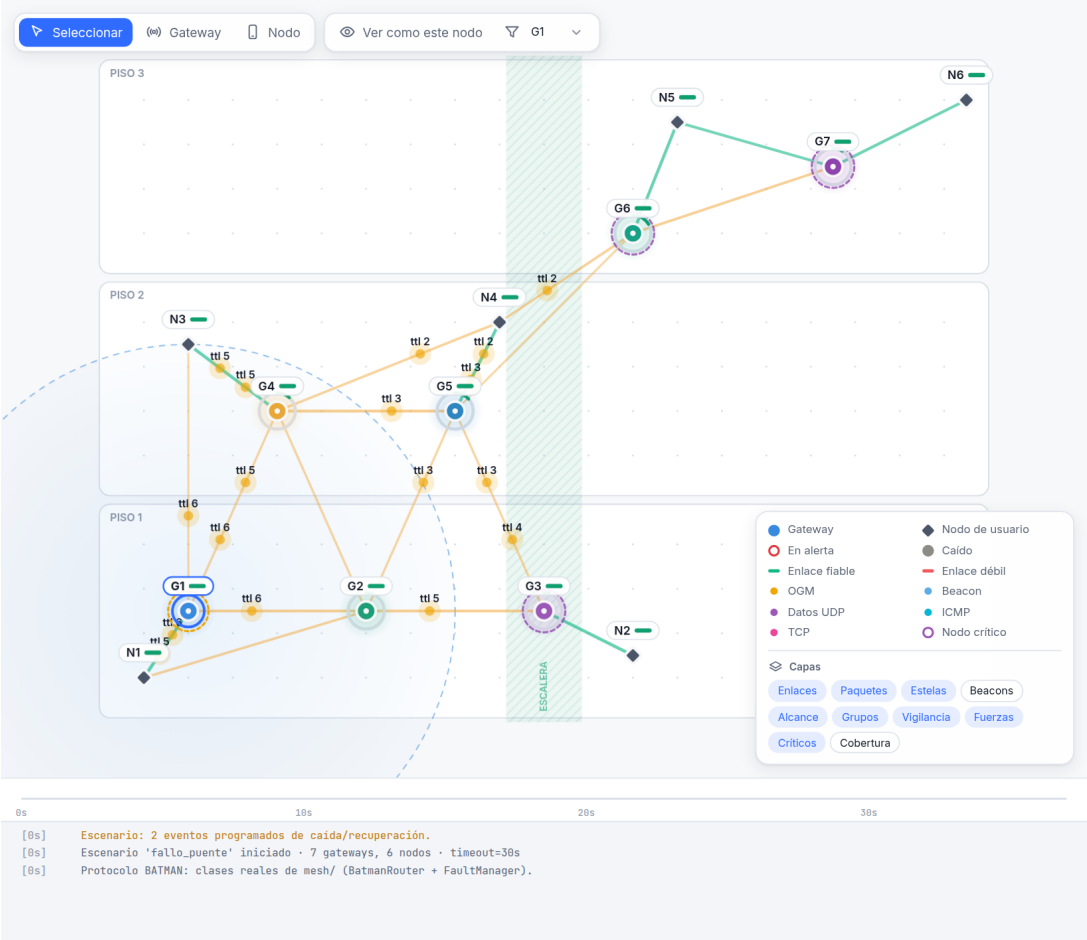


Figura 5.2: El protocolo en funcionamiento: propagación de los OGM originados por G1 (captura de la interfaz, escenario `fallo_puente`, semilla 1). Cada punto naranja es una copia del mensaje viajando por un enlace, con el número de saltos que le quedan ("ttl"). Las copias cercanas a G1 llevan ttl 6 y 5; las que ya cruzaron varios nodos, ttl 3 y 2. El círculo punteado es el alcance de radio de G1

UNIVERSIDAD SERGIO ARBOLEDA

- w es cada elemento de la ventana: vale 1 si esa secuencia se recibió y 0 si se perdió;
- $LQ_{o,v}$ es la calidad del enlace: la fracción de OGM recibidos, entre 0 y 1;
- TQ_{OGM} es la calidad que trae escrita el mensaje, ya reducida por cada nodo que lo reenvió;
- $TQ_{o,v}$ es la calidad de la ruta hacia o a través de v, entre 0 y 1.

Elección de ruta. Para cada destino, el nodo guarda el vecino con mejor TQ. Otro vecino lo reemplaza solo si su TQ es mayor que el del actual, una vez descontados los mensajes que el actual ha dejado de entregar.

Detección de fallos. Cada Gateway revisa cada 5 s, con `FaultManager`, cuánto hace que no oye a los demás, y marca como caído al que lleva más de 30 s en silencio. Nadie coordina esta revisión. Un mensaje nuevo del nodo apaga la alerta.

5.2.3. El modelo del medio de radiofrecuencia

`RadioMedium` es lo único que reemplaza a la red real. Cada vez que un nodo transmite, el modelo decide por separado, para cada nodo vivo, si el mensaje le llega. Lo hace al azar, con la probabilidad de la Ecuación (5.2), que se llama aquí *fiabilidad* del enlace:

$$r(d,\Delta f) = \begin{cases} \max\!\big(0,\ \min\!\big(1,\ [\,1-\phi\,d/R-p_0\,]\ a_f^{\Delta f}\big)\big) & \text{si } d \le R \\ 0 & \text{si } d > R \end{cases} \tag{5.2}$$

donde:

- r es la fiabilidad: la probabilidad, entre 0 y 1, de que una transmisión llegue;
- d es la distancia entre los dos nodos, en metros;
- Δf es el número de pisos de diferencia entre los dos nodos;
- R es el alcance máximo del radio, en metros: más lejos no llega nada;
- ϕ es la degradación con la distancia: cuánto baja la fiabilidad al llegar al borde del alcance;
- p_0 es la pérdida base: la fracción de mensajes que se pierde aun a distancia cero;
- a_f es la atenuación por piso: el factor, entre 0 y 1, por el que se multiplica la fiabilidad por cada piso de diferencia.

UNIVERSIDAD
SERGIO ARBOLEDA

En palabras: la fiabilidad baja en línea recta con la distancia, se reduce por cada piso que separa a los nodos y es cero fuera del alcance. Que cada piso reste señal es un efecto medido en edificios reales [Seidel and Rappaport, 1992]. La Tabla 5.2 muestra los valores usados. El modelo es simple a propósito: es simétrico, no cambia en el tiempo y no representa colisiones, supuestos que se saben optimistas [Kotz et al., 2004]. Lo que aporta frente a un simple dibujo de conexiones es la pérdida entre pisos y que los mensajes perdidos hagan variar el TQ. La distancia se mide en el corte vertical del edificio: es un modelo de dos dimensiones con pisos, no una propagación en tres dimensiones.

Cuadro 5.2: Valores de los parámetros del medio y del protocolo: los que el simulador trae por defecto, los del despliegue de referencia en el edificio y los del escenario de campo abierto

Parámetro	**Por defecto**	**Despliegue**	**Campo abierto**
Alcance de radio R (m)	16	12	30
Degradación con la distancia ϕ	0.85	0.5	0.6
Pérdida base p_0	0.02	0.02	0.02
Atenuación por piso a_f	0.55	0.8	no aplica
Tiempo límite de detección (s)	30	30	30
Periodo de *beacons* / de OGM (s)	2 / 4	2 / 4	2 / 4
Velocidad de un Gateway (m por paso)	0.32	0 (fijos)	0.6

5.2.4. Envío de paquetes de datos

En el simulador de partida, un mensaje se entregaba buscando el camino en la red real, y llegaba siempre que existiera alguno. Las tablas de rutas no decidían nada. En este diseño, cada paquete avanza salto a salto: en cada nodo, el siguiente salto lo decide `BatmanRouter.next_hop()` con la tabla de ese nodo. Nadie conoce el camino completo. Cada salto es un envío dirigido a un solo vecino, que se reintenta hasta 3 veces si se pierde, como hace la capa de enlace de las redes inalámbricas [IEEE, 2021]. Un paquete que da más de 50 saltos se descarta.

Así, un paquete puede perderse aunque exista un camino, y la causa queda registrada. Las pérdidas se separan en físicas (el destino está caído o no hay ningún camino por radio) y de enrutamiento (el nodo no tiene ruta, la ruta apunta a un vecino que ya no está al alcance, el radio perdió el salto o el paquete quedó dando vueltas en un bucle). Esta separación permite no culpar al protocolo de lo que es física. Cada salto se resuelve dentro del mismo paso, porque un salto real dura mucho menos que 0.5 s; por eso el simulador no mide el retardo.

Tipos de paquete. El usuario elige qué tipo de paquete se envía, cuántos y cada cuánto tiempo. Los tres tipos usan el mismo avance salto a salto y se diferencian en lo

UNIVERSIDAD SERGIO ARBOLEDA

que ocurre entre el origen y el destino:

- **UDP** [Postel, 1980]: un paquete de ida, sin confirmación. Si se pierde, nadie lo vuelve a enviar.
- **ICMP** [Postel, 1981]: un eco, como la orden `ping`. La petición va y el destino responde, usando su propia tabla de rutas para el regreso. Cuenta como entregado solo si la respuesta vuelve al origen.
- **TCP** [Eddy, 2022]: el destino confirma cada paquete y el origen lo vuelve a enviar, hasta 3 veces, si la confirmación no llega.

Son versiones simplificadas de esos protocolos: no hay establecimiento de conexión ni control de congestión, y los reenvíos de TCP ocurren dentro del mismo paso, de modo que solo recuperan pérdidas del radio.

5.2.5. Movimiento descentralizado de los Gateway

El simulador tiene tres modos de movimiento de los Gateway. Dos deciden con información global y se conservan solo como referencia: *seguir* lleva a cada Gateway hacia el usuario más cercano mirando las posiciones de todos, y *repartir* asigna un usuario distinto a cada Gateway desde el motor del simulador.

En el modo *descentralizado*, cada Gateway decide únicamente con lo que oyó en los *beacons* de sus vecinos, que llevan la posición de quien los emite. Esto supone que cada nodo conoce su posición, por ejemplo con un sistema de localización, como es habitual en estos métodos [Howard et al., 2002]. Si pasa 3 periodos sin oír a un vecino, lo olvida. La decisión combina cuatro reglas, de la familia de las "fuerzas virtuales" revisadas en la Sección 3.6 [Howard et al., 2002, Zou and Chakrabarty, 2003, Poduri and Sukhatme, 2004], con distancias expresadas como fracción del alcance R:

- **Cobertura.** El Gateway se acerca a los usuarios que oye y de los que es el Gateway más cercano, si están a más de $0{,}4R$.
- **Separación.** Se aleja de los Gateway que oye a menos de $0{,}55R$, para no amontonarse.
- **Expansión.** Si no tiene usuarios a su cargo, se aleja de los demás hasta $0{,}75R$: la malla se estira hasta oír usuarios nuevos.
- **Reconexión.** Si no oye a ningún Gateway, vuelve hacia el último que oyó.

A estas reglas se suma una de seguridad: un Gateway no da un paso que rompa el enlace con otro Gateway que oye, salvo que quede un camino de dos saltos a través de un tercero. Como la posición que trae un *beacon* puede tener varios segundos, el Gateway supone que el vecino pudo haberse movido y solo considera seguro un enlace si lo es en el peor caso. La Figura 5.8 muestra el movimiento descentralizado en la interfaz.

UNIVERSIDAD SERGIO ARBOLEDA

Los umbrales se eligieron probando combinaciones sobre cuatro escenarios, con 5 semillas. Esa búsqueda dejó dos aprendizajes. Sin la regla de expansión, los Gateway que arrancan juntos y no oyen a nadie se quedan quietos: en el escenario `base` la cobertura era de 0.23. Y recordar a los vecinos por más tiempo empeoraba las particiones, porque las posiciones viejas engañan más de lo que ayudan.

5.2.6. Medir lo que cree cada nodo

Para cada par de nodos vivos, el simulador compara en cada muestra lo que cree el primero (si tiene una ruta vigente hacia el segundo) con la realidad (si el radio los conecta). De ahí salen la exactitud del conocimiento y las rutas fantasma y faltantes de la Tabla 4.2. Una ruta es *vigente* si un OGM la refrescó hace menos que el tiempo límite.

Un *nodo crítico* es aquel cuya caída parte la red en dos o más grupos. Se calculan con el algoritmo de puntos de articulación de un grafo [Hopcroft and Tarjan, 1973]. Con esa base se diseñaron dos tipos de caída que eligen al nodo en el momento del fallo: *al azar* y *dirigida*, que tumba al nodo que más nodos deja aislados. Esto permite comparar un fallo cualquiera con el peor fallo posible, como en los estudios de errores y ataques en redes [Albert et al., 2000].

5.2.7. Otros mapas, otros movimientos y un protocolo de comparación

Para que la herramienta no quede atada al edificio de partida, el diseño incluye:

- **Mapa de campo abierto:** un área vista desde arriba, sin pisos. El escenario `campo_abierto` mide 120 × 80 m y tiene 6 Gateway que salen del mismo punto, 9 usuarios en 3 grupos y 6 flujos de datos.
- **Movimiento de los usuarios:** además de usuarios quietos, pueden caminar hacia puntos al azar (*random waypoint* [Camp et al., 2002]) o moverse en grupos (inspirado en el modelo RPGM [Hong et al., 1999]).
- **Protocolo de comparación:** en la *inundación*, cada nodo repite una vez cada paquete nuevo a todos sus vecinos, sin tablas de rutas ni reintentos [Ni et al., 1999].
- **Parámetros en un solo lugar:** cualquier parámetro del medio o del protocolo se puede cambiar desde la terminal, un lote o la interfaz.

5.3. Implementación

El simulador está escrito en el lenguaje Python [Python Software Foundation, 2026]. Para los cálculos y las figuras usa las bibliotecas NumPy [Harris et al., 2020] y Mat-

SERGIO ARBOLEDA

plotlib [Hunter, 2007], y para la ventana propia, pygame-ce [Pygame Community, 2026]. La interfaz web usa solo lo que trae Python y páginas que el navegador abre sin instalar nada más.

5.3.1. Escenarios, lotes y formas de ejecución

Un escenario es un archivo de texto que describe el medio, el protocolo, los nodos, los eventos y el tráfico. Al cargarlo, el simulador lo revisa y rechaza con un mensaje claro los errores, como identificadores repetidos o posiciones fuera del mapa. Este es un fragmento del escenario de la caída del puente, con un flujo de tráfico añadido como ejemplo:

```
[medium]
rango_comm=12        # alcance R, en metros
falloff=0.5          # degradación con la distancia
floor_atten=0.8      # atenuación por piso

[nodes]
# id  rol  x   y   (G = Gateway, N = usuario)
1    G    4   5
...
6    G    24  22
101  N    2   2

[events]
fail     6  60       # el nodo 6 se cae a los 60 s
recover  6  150      # y vuelve a los 150 s

[traffic]
# origen destino cada(s) tipo cantidad
101  104  1  tipo=icmp cantidad=20
```

La misma simulación se puede ejecutar de cuatro formas: en una ventana propia; sin ventana, con el resumen en la terminal; por lotes, que repiten varios escenarios con varias semillas y calculan el promedio; y en la interfaz web. Las cuatro construyen la simulación con la misma función, de modo que dan los mismos números.

5.3.2. Interfaz web

La interfaz se abre con `python main.py --web`, funciona sin conexión a Internet y la atiende el mismo programa del simulador. El navegador solo dibuja lo que recibe: la fiabilidad, las rutas y las métricas se calculan en el simulador, de modo que no hay una segunda versión de los cálculos que pueda dar otro resultado. Esta sección muestra

UNIVERSIDAD SERGIO ARBOLEDA

cada función de la interfaz con una captura tomada del simulador en funcionamiento. Todas las capturas son de elaboración propia.

Vista general. La Figura 5.3 muestra la pantalla completa durante la prueba de la caída del puente. A la izquierda está el mapa del edificio con los nodos y sus enlaces; arriba, los controles de tiempo y el estado de la red; a la derecha, el panel del nodo seleccionado, con su tabla de rutas; abajo, la línea de tiempo y el registro de lo que va ocurriendo.

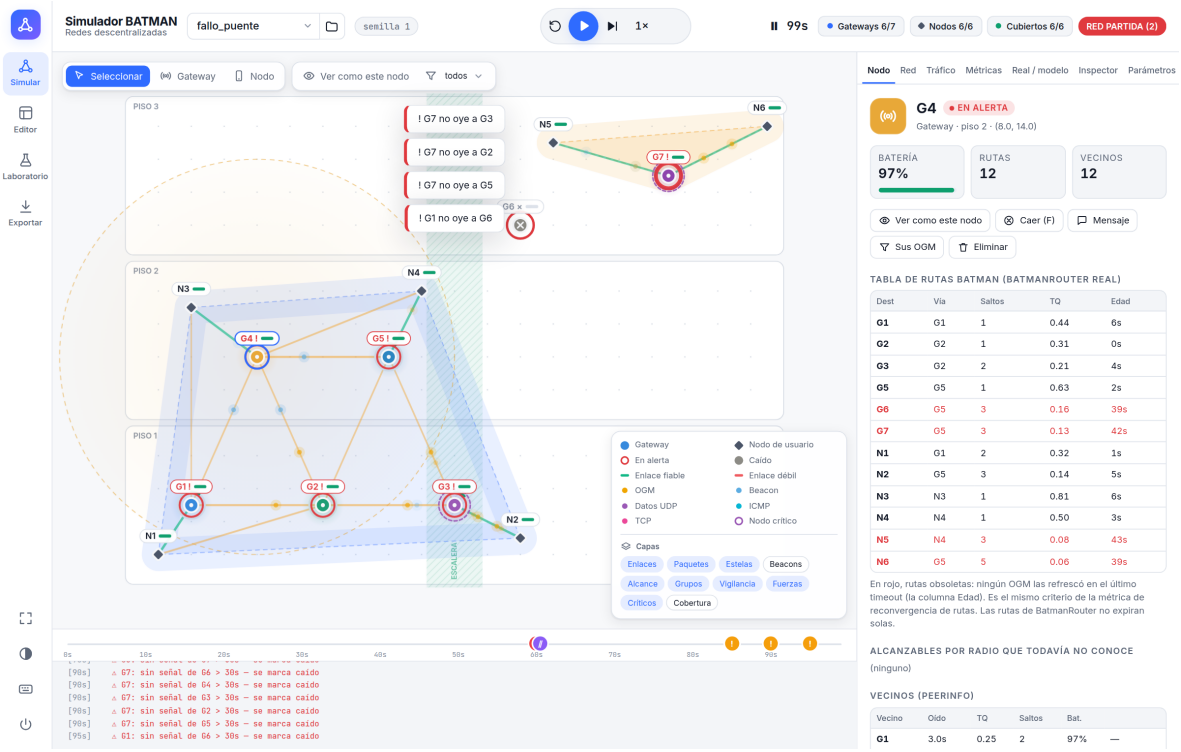


Figura 5.3: Vista general de la interfaz a los 99 s de la prueba de la caída del puente (semilla 1). En el mapa, G6 aparece caído (círculo gris con una equis) y las dos zonas sombreadas son los dos grupos en que quedó partida la malla. Arriba a la derecha, el aviso rojo "Red partida (2)" y el indicador "Cubiertos 6/6": los seis usuarios siguen cubiertos. Los Gateway con borde rojo tienen alertas activas. A la derecha, la tabla de rutas de G4: en rojo, las rutas hacia el piso 3 que ya nadie refresca

Lo que sabe un nodo. El panel de nodo (a la derecha en la Figura 5.3) muestra la tabla de rutas del nodo: para cada destino, por cuál vecino enviar, a cuántos saltos está, con qué calidad y hace cuánto se refrescó. La función "Ver como este nodo" (Figura 5.4) redibuja el mapa solo con lo que sabe el nodo elegido.

Lo que sabe toda la red. La matriz de conocimiento (Figura 5.5) reúne en una cuadrícula lo que sabe cada nodo sobre cada uno de los demás, y permite ver de un vistazo si la red ya se conoce a sí misma y qué rutas quedan después de una partición.

Qué es real y qué es modelo. El panel de la Figura 5.6 atiende el requerimiento

SERGIO ARBOLEDA

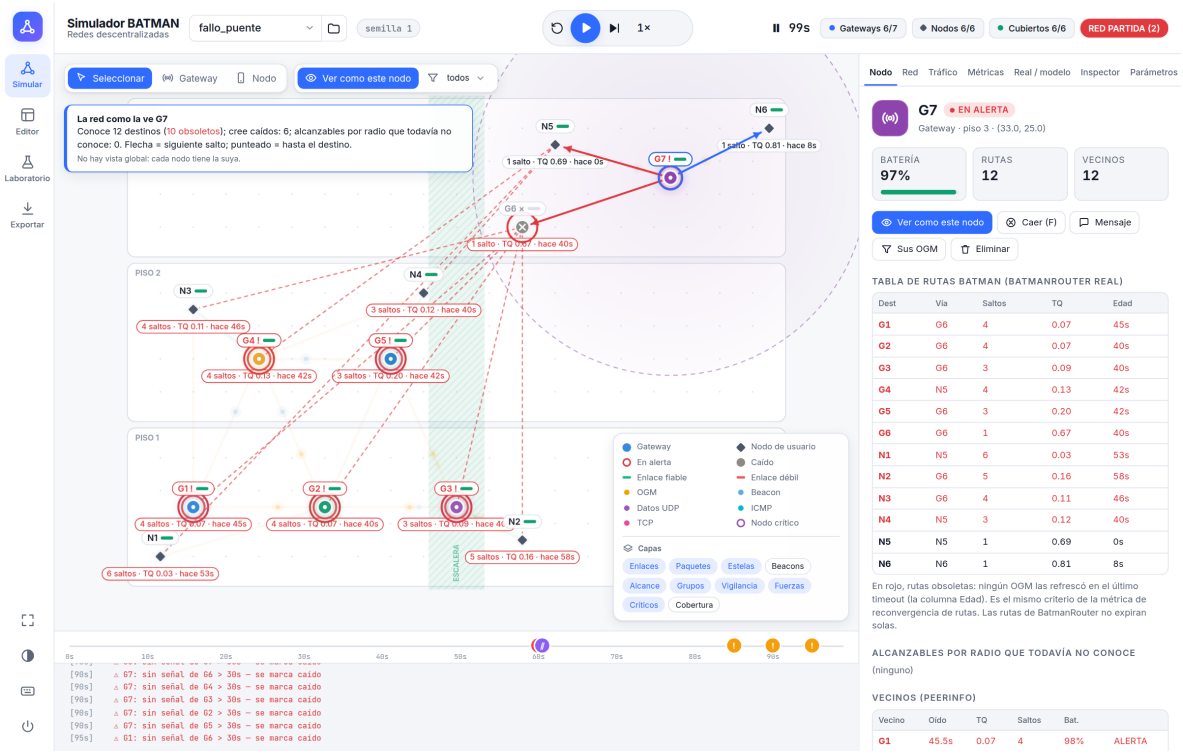
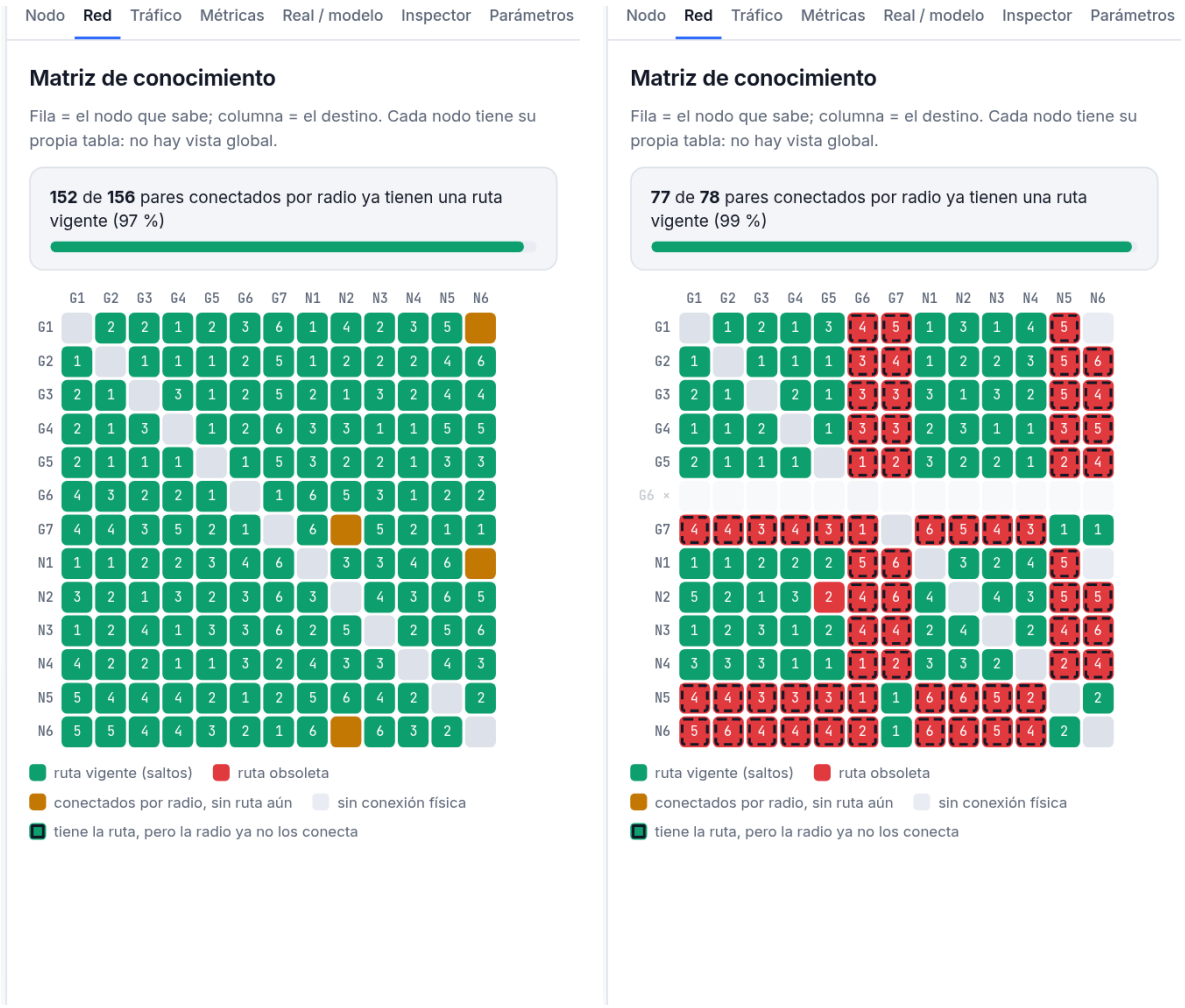


Figura 5.4: Función “Ver como este nodo”, aplicada a G7 a los 99 s de la prueba de la caída del puente. El mapa solo muestra lo que G7 sabe: las flechas continuas son sus rutas vigentes (hacia N5 y N6) y las líneas punteadas rojas son rutas que conserva en su tabla hacia nodos que ya no puede alcanzar, con los saltos, la calidad y la antigüedad de cada una. El recuadro superior resume cuántos destinos conoce y cuántos están obsoletos

UNIVERSIDAD
SERGIO ARBOLEDA



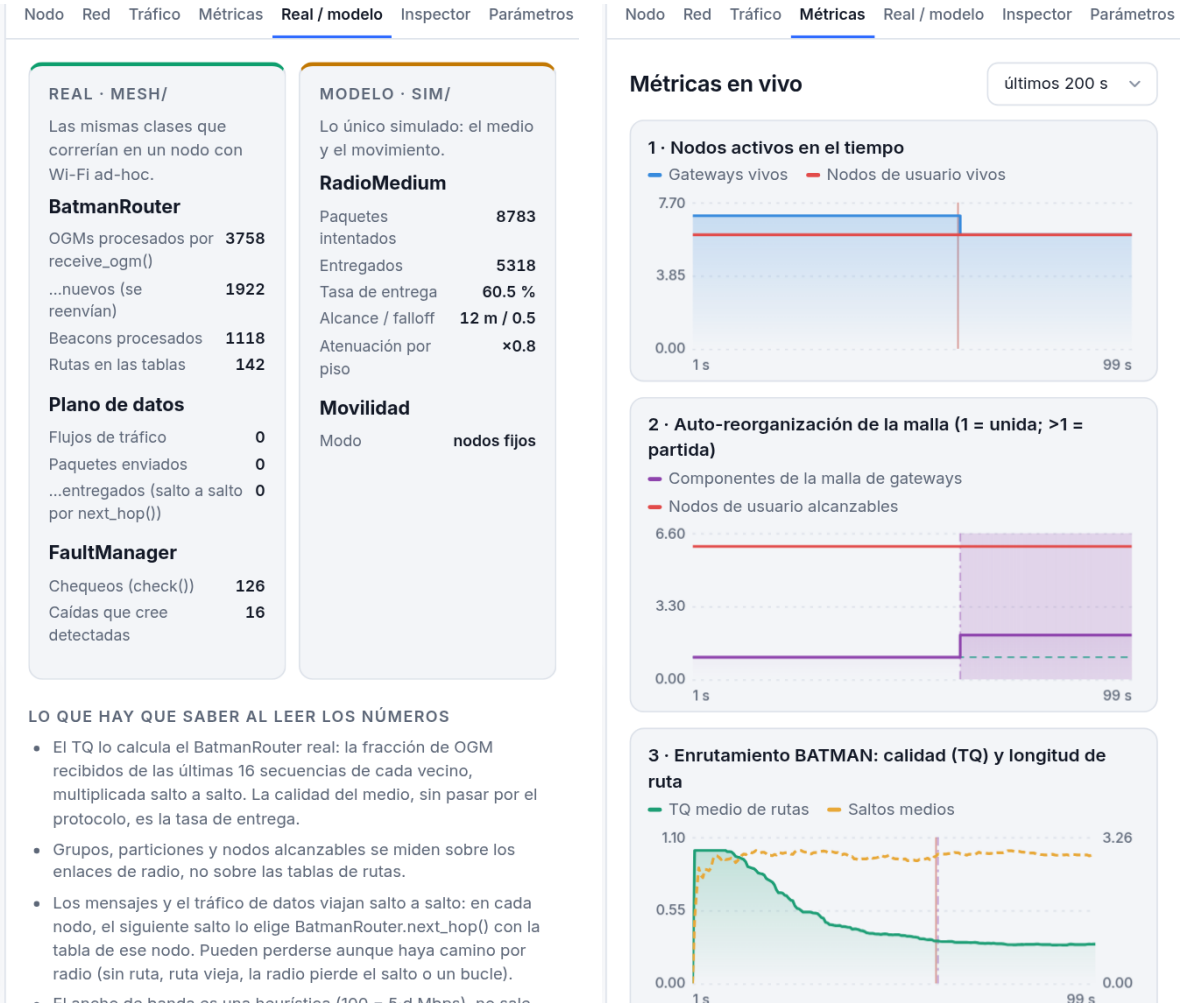
(a) A los 30 s, antes del fallo

(b) A los 99 s, con el puente caído

Figura 5.5: Matriz de conocimiento. Cada fila es un nodo que "sabe" y cada columna, un destino. Una casilla verde es una ruta vigente, con su número de saltos; una naranja, un par conectado por radio que todavía no tiene ruta; una roja, una ruta obsoleta. (a) Con la malla formada, 152 de los 156 pares conectados por radio ya tienen ruta (97 %). (b) Con G6 caído, las rutas entre el piso 3 y el resto aparecen en rojo: siguen en las tablas, pero ya no sirven

UNIVERSIDAD SERGIO ARBOLEDA

R2: separa lo que hizo el código del nodo (mensajes procesados, rutas, revisiones de fallos) de lo que aportó el modelo (transmisiones intentadas y entregadas por el medio).



(a) Panel "Real / modelo" (b) Panel de métricas en vivo

Figura 5.6: Dos paneles de la interfaz a los 99 s de la prueba de la caída del puente. (a) A la izquierda, lo que ejecutó el código del nodo (OGM y *beacons* procesados, rutas en las tablas, revisiones de fallos); a la derecha, lo que aportó el modelo: el medio intentó 8783 entregas y completó el 60.5 %. (b) Gráficas que se actualizan durante la corrida: nodos vivos, número de grupos de la malla (sube de 1 a 2 al caer el puente, zona sombreada) y calidad y longitud de las rutas

Tráfico de datos. En el panel de la Figura 5.7 se eligen el origen, el destino, el tipo de paquete, la cantidad y el intervalo. El panel muestra cuántos paquetes de cada tipo se enviaron y se entregaron, los flujos activos y el recorrido de los últimos paquetes, incluida la respuesta de regreso en ICMP y TCP.

Capas del mapa. El mapa tiene capas que se encienden y se apagan. La Figura 5.8

UNIVERSIDAD SERGIO ARBOLEDA

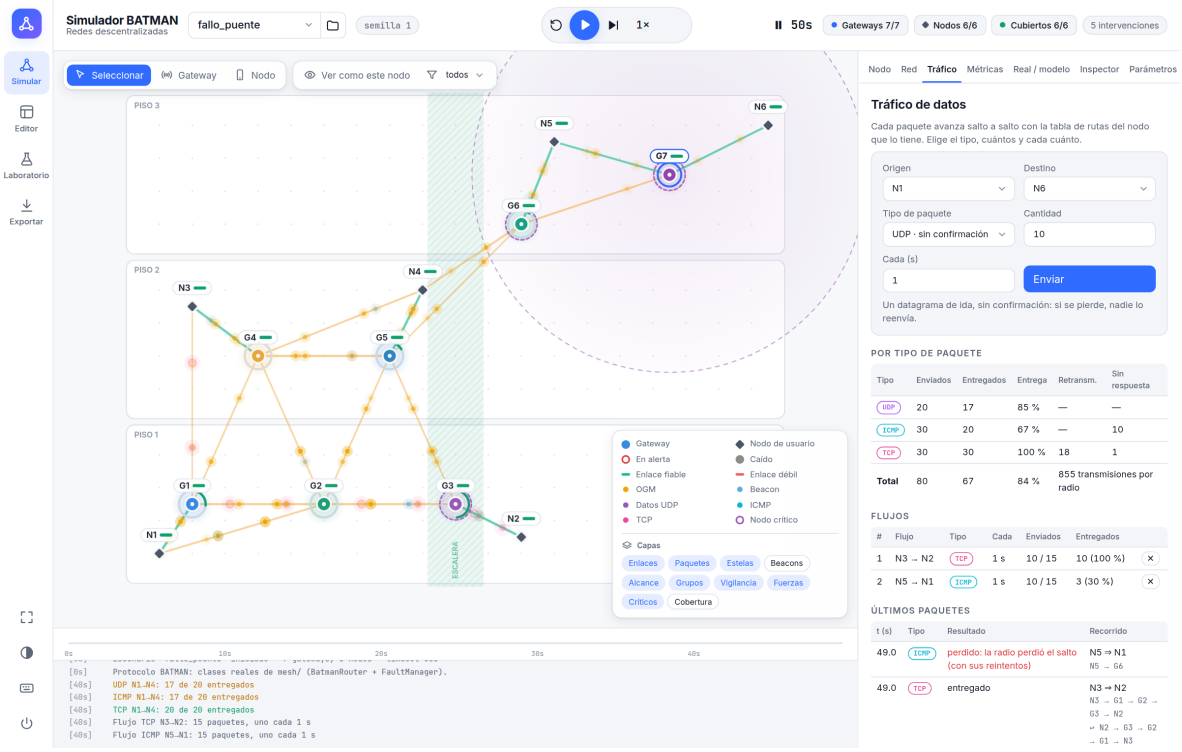


Figura 5.7: Panel de tráfico a los 50 s de una sesión sobre el despliegue de referencia. Arriba, el formulario para elegir origen, destino, tipo de paquete, cantidad e intervalo. En la tabla “Por tipo de paquete” se ve el resultado de enviar 20 paquetes de cada tipo de N1 a N4 y dos flujos adicionales: UDP entregó 17 de 20; ICMP, 20 de 30, con 10 ecos sin respuesta; TCP, 30 de 30, con 18 reenvíos. Abajo, los flujos en curso y los últimos paquetes con su recorrido de ida y de vuelta. En el mapa, los puntos de colores son paquetes en tránsito

UNIVERSIDAD
SERGIO ARBOLEDA

muestra tres de ellas con Gateway móviles: el mapa de cobertura, los nodos críticos y la decisión de movimiento de cada Gateway.

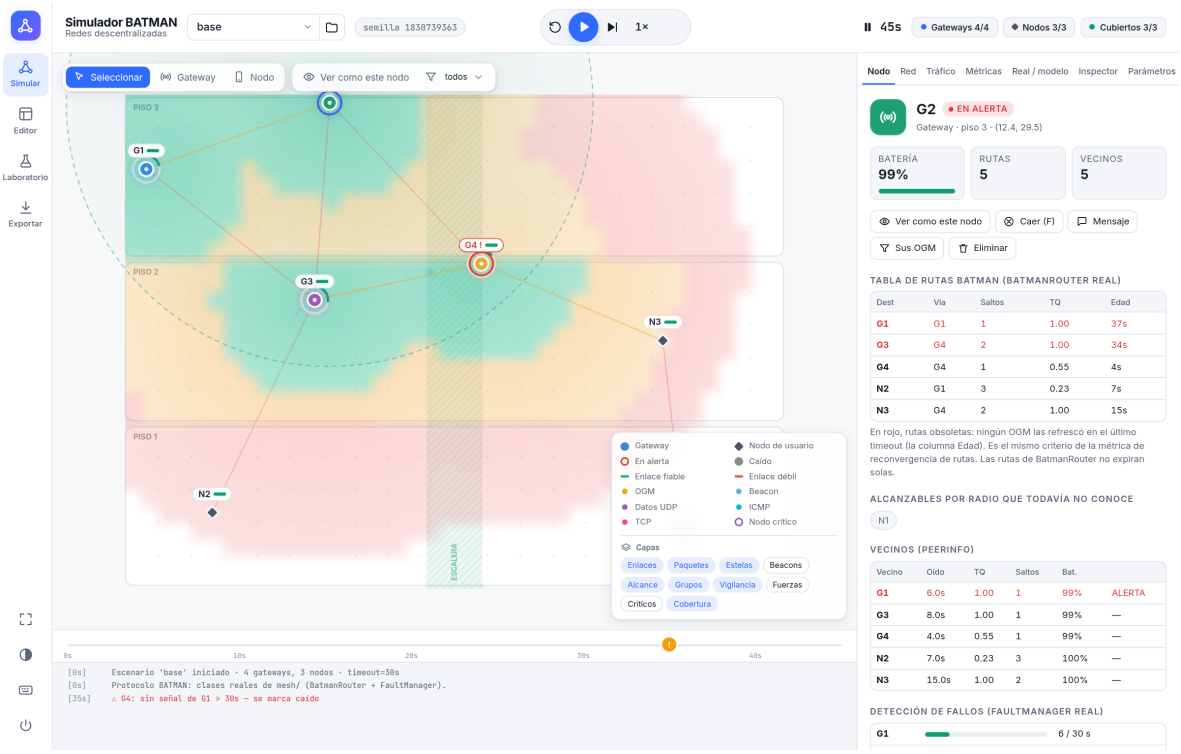


Figura 5.8: Capas del mapa con movimiento descentralizado (escenario `base` a los 45 s). El fondo de colores es el mapa de cobertura: verde donde la señal de algún Gateway es buena, amarillo donde es débil y rojo donde no llega. Los Gateway se han separado desde su posición inicial para cubrir más área; el anillo punteado marca un nodo crítico y el círculo grande, el alcance del Gateway seleccionado (G2), cuya tabla de rutas y vecinos aparece a la derecha

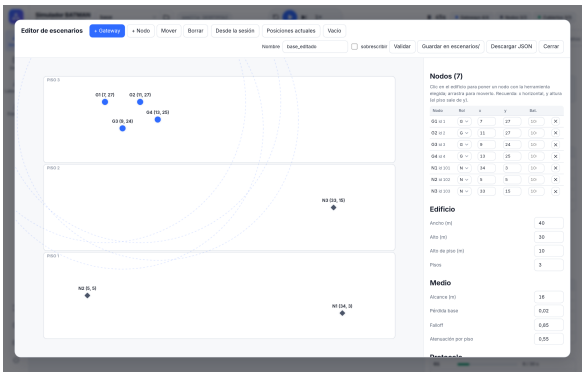
Editor de escenarios y laboratorio. El editor (Figura 5.9a) permite armar un escenario con el ratón y guardarlo como archivo. El laboratorio (Figura 5.9b) corre lotes de escenarios con varias semillas, con los mismos números que la terminal.

La Tabla 5.3 resume las funciones de la interfaz, la figura que sirve de evidencia de cada una y el requerimiento que atiende.

5.3.3. Costo por nodo

La Tabla 5.4 mide la velocidad del simulador con redes de distinto tamaño en el mismo edificio, con nodos fijos y 6 flujos de datos, en un computador portátil con un procesador Intel Core i5-1035G1. El costo crece mucho más rápido que el número de nodos: al medir la corrida de 100 nodos, cerca del 90 % del tiempo se fue en procesar y reenviar los OGM, porque al crecer la red en la misma área todos se oyen con todos y

UNIVERSIDAD SERGIO ARBOLEDA



(a) Editor de escenarios (b) Laboratorio de experimentos

Figura 5.9: (a) Editor de escenarios: a la izquierda, el edificio donde se ponen y se arrastran los nodos; a la derecha, la lista de nodos y los valores del edificio y del medio. (b) Laboratorio: se indica la duración, el número de semillas y los escenarios que se quieren comparar, y al terminar muestra el promedio de cada métrica

Cuadro 5.3: Funciones de la interfaz, figura que muestra cada una y requerimiento de la Tabla 5.1 que atiende

Función	**Qué permite**	**Evidencia**	**Req.**
Panel de nodo	Ver la tabla de rutas y los vecinos de un nodo	Figura 5.3	R1, R5
“Ver como este nodo”	Ver la red solo con lo que sabe un nodo	Figura 5.4	R4, R5
Matriz de conocimiento	Ver qué sabe cada nodo de los demás	Figura 5.5	R5
OGM de un origen, con su TTL	Seguir los mensajes del protocolo salto a salto	Figura 5.2	R1
Panel “Real / modelo”	Separar lo que hizo el código de lo que aportó el modelo	Figura 5.6a	R2
Métricas en vivo	Ver la evolución de la red durante la corrida	Figura 5.6b	R5
Panel de tráfico	Elegir tipo, cantidad e intervalo de los paquetes y ver su recorrido	Figura 5.7	R9
Capas de cobertura, críticos y fuerzas	Ver el efecto de la distancia y los pisos, y la decisión de cada Gateway	Figura 5.8	R3, R4
Caer y recuperar nodos, arrastrarlos y cambiar parámetros	Experimentar sin equipos físicos	Figura 5.3	R8, R10
Editor y laboratorio	Crear escenarios y repetirlos con semillas	Figura 5.9	R6, R10

UNIVERSIDAD
SERGIO ARBOLEDA

cada OGM se repite muchas veces. Esto marca el rango de uso de la herramienta: con decenas de nodos corre más rápido que el tiempo real; con unos 100 nodos juntos, más lento; y no sirve para miles de nodos sin optimizarla. El requerimiento R7 se cumple entonces solo en parte: se evita el costo de una máquina virtual por nodo, pero un emulador como CORE reporta más de un centenar de nodos en tiempo real [Ahrenholz et al., 2008]. La ventaja de esta herramienta frente a la emulación está en el control, la repetibilidad y las mediciones, no en el tamaño.

Cuadro 5.4: Velocidad del simulador según el tamaño de la red (una corrida, semilla 1). La última columna indica cuántos segundos de simulación se calculan en un segundo de reloj: por encima de 1, el simulador va más rápido que el tiempo real

Nodos	**Gateway**	**Tiempo por paso (ms)**	**Segundos simulados por segundo real**
14	8	1.8	284.5
25	10	9.2	54.1
50	20	78.6	6.4
100	40	816.7	0.6
200	80	12 006	0.04

5.3.4. Diferencias con B.A.T.M.A.N.

Conviene precisar qué significa aquí "código real". `BatmanRouter` y `FaultManager` son las clases que usa el nodo de red de este proyecto (`mesh/node.py`), de modo que lo que se evalúa es esa implementación y no una copia escrita aparte. Pero esa implementación no es el módulo batman-adv [Open Mesh, 2026], ni se ha ejecutado en este trabajo sobre equipos de radio. Las diferencias conocidas con el protocolo [Neumann et al., 2008] son:

- **Reenvío de OGM.** Se reenvía la primera copia de cada mensaje, llegue por donde llegue. El protocolo solo reenvía la que llegó por el mejor vecino. Esto produce bucles en las tablas (Sección 6.9).
- **Vencimiento de rutas.** Las rutas no se borran solas: una ruta por un vecino caído solo se reemplaza cuando aparece otra mejor. El protocolo borra al nodo de origen cuando pasa un tiempo sin oírlo.
- **Reinicio.** Un nodo que se recupera conserva su número de secuencia; un arranque desde cero queda fuera del modelo.

Durante el desarrollo se corrigieron además tres defectos que afectaban a las conclusiones: el TQ valía siempre 1.0 porque nunca se registraban los OGM perdidos, la primera copia de cada mensaje ganaba la ruta sin mirar el TQ, y una alerta sobre un Gateway lejano nunca se apagaba.

UNIVERSIDAD SERGIO ARBOLEDA

5.4. Pruebas

Las pruebas del simulador son simulaciones: escenarios que cualquiera puede volver a ejecutar, cada uno con un propósito y un resultado esperado que se compara con lo que el simulador produce. La Tabla 5.5 las presenta. Las pruebas P1 a P3 comprueban funciones básicas con una corrida que se puede observar en la interfaz; las pruebas P4 a P9 comparan configuraciones con 10 semillas cada una, y las pruebas P10 y P11 comprueban propiedades de la herramienta. Los resultados están en el Capítulo 6.

Cuadro 5.5: Pruebas de simulación: nombre, propósito y resultado esperado de cada una

N.º	**Nombre**	**Propósito**	**Qué se espera**
P1	Formación de la malla	Comprobar que los nodos se descubren y arman sus tablas de rutas sin ayuda	A los 30 s, casi todos los pares de nodos conectados por radio tienen una ruta
P2	Propagación de un OGM	Comprobar que los mensajes del protocolo se reenvían salto a salto	El TTL de las copias baja en uno por cada salto desde el origen
P3	Tipos de paquete	Comparar la entrega de UDP, ICMP y TCP con el mismo tráfico (lote `tipos_paquete`)	ICMP entrega menos que UDP, porque necesita ida y vuelta; TCP entrega más que UDP, con más transmisiones
P4	Fallos con Gateway fijos	Medir cómo responde la malla a la caída de un Gateway redundante, de un puente, de uno de borde y a una cascada de caídas (lote `capitulo5_fallos`)	El redundante no afecta la cobertura; el puente parte la malla 90 s; el de borde deja a N6 sin cobertura (0.9375); la cascada baja la cobertura a 0.736
P5	Los mismos fallos con Gateway móviles	Medir si la malla se adapta sola cuando los Gateway pueden moverse (lote `capitulo5_fallos_movil`)	La malla pasa menos tiempo partida que con Gateway fijos
P6	Modos de movimiento	Comparar el movimiento descentralizado con los dos modos centralizados en 5 escenarios (lote `movilidad`)	El modo descentralizado mantiene la malla unida con una cobertura cercana a la de los centralizados
P7	Movimiento con tráfico	Medir qué significan esas diferencias para los paquetes de datos, en 3 escenarios con 6 flujos (lote `movilidad_datos`)	El modo que conecta a más nodos entrega más paquetes
P8	Fallos al azar y dirigidos	Comparar dos caídas al azar con dos dirigidas al nodo más crítico (lote `robustez`)	Los fallos dirigidos hacen más daño que los fallos al azar

UNIVERSIDAD
SERGIO ARBOLEDA

N.º	Nombre	Propósito	Qué se espera
P9	B.A.T.M.A.N. e inundación	Comparar el protocolo con la inundación en campo abierto y en el edificio (lote `generalidad`)	La inundación entrega más a cambio de más transmisiones
P10	Repetibilidad	Comprobar que la misma semilla da la misma corrida, también en otro computador	Las tablas de resultados salen idénticas
P11	Tamaño de la red	Medir hasta cuántos nodos es práctico el simulador	El tiempo de cálculo crece con el número de nodos (Tabla 5.4)

En las pruebas de fallos (P4 y P5), el Gateway redundante (G2), el puente (G6) y el de borde (G7) se caen a los 60 s y vuelven a los 150 s. En la cascada se caen G6 a los 60 s, G7 a los 100 s y G3 a los 140 s, y no vuelven. Las corridas duran entre 200 y 300 s simulados.

5.4.1. Ejemplo de una prueba: la caída del puente

Para mostrar cómo se lee una prueba, se toma una corrida de la prueba P4: la caída del Gateway puente con la semilla 1. Se ejecuta con la orden:

```
python main.py --headless --duracion 240 --seed 1 \
    --config escenarios/fallos/fallo_puente.txt
```

El resultado esperado se puede calcular antes de simular: G6 es el único enlace con el piso 3 y está caído de los 60 s a los 150 s, de modo que la malla debe estar partida exactamente 90 s; y como cada parte conserva algún Gateway, los seis usuarios deben seguir cubiertos. La Tabla 5.6 muestra lo que entrega el simulador al terminar, y la Figura 5.3 muestra la misma corrida en la interfaz a los 99 s: la red partida en dos y los seis usuarios cubiertos. Las dos cosas coinciden con lo esperado.

5.4.2. Comprobaciones automáticas del código

Además de las pruebas de simulación, el repositorio incluye 390 comprobaciones automáticas, que se ejecutan con una sola orden y avisan si un cambio en el programa alteró algo que ya funcionaba. No son el tema de esta sección; su detalle está en el Anexo B.

UNIVERSIDAD
SERGIO ARBOLEDA

Cuadro 5.6: Métricas de una corrida de la prueba de la caída del puente (escenario `fallo_puente`, 240 s simulados, semilla 1), tal como las entrega el simulador al terminar

Métrica	**Valor**	**Qué se esperaba**
Tiempo con la malla partida (s)	90.0	90 (lo que dura la caída)
Cobertura media de los Nodos de usuario	1.000	1 (nadie queda aislado)
Nodos de usuario alcanzables al final	6	6
Gateway vivos al final	7	7 (G6 vuelve)
Grupos de la malla al final	1	1 (se reunifica)
Reconvergencia de rutas (s)	7.5	Unos pocos periodos de OGM
Alertas de Gateway perdido	22	Varias: los demás dejan de oír a G6
Instante de la primera alerta (s)	85.0	Entre 60 y 90 (límite de 30 s)
TQ medio de las rutas	0.344	Sin valor esperado
Saltos medios por ruta	2.83	Sin valor esperado
Tasa de entrega del radio	0.604	Sin valor esperado
Exactitud del conocimiento de rutas	0.912	Menor que 1 durante la partición
Pares con ruta fantasma / faltante	0.038 / 0.050	Mayores que 0 durante la partición
Nodos críticos, promedio	2.62	Sin valor esperado

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 6

Resultados y discusión

Este capítulo presenta los resultados de las pruebas de la Tabla 5.5, en el mismo orden. Para cada prueba se recuerda qué se esperaba y se compara con lo obtenido; la Tabla 6.11, al final, las resume. Salvo que se indique otra cosa, las cifras son el promedio ± la desviación estándar de 10 semillas, calculadas directamente a partir de los lotes. Las figuras y tablas de este documento que no indican fuente son de elaboración propia.

6.1. Funciones básicas (pruebas P1 y P2)

P1, formación de la malla. Se esperaba que, sin ninguna ayuda, los nodos se descubrieran y llenaran sus tablas de rutas. A los 30 s de la corrida con la semilla 1, 152 de los 156 pares de nodos conectados por radio ya tenían una ruta vigente, es decir, el 97 % (Figura 5.5a). Los cuatro pares que faltan son pares lejanos, entre el piso 1 y el piso 3: sus OGM deben cruzar muchos saltos de enlaces con pérdidas, cerca del límite de 6 saltos del TTL. La prueba se cumple.

P2, propagación de un OGM. Se esperaba que el TTL bajara en uno por cada salto. En la Figura 5.2, las copias del mensaje de G1 llevan ttl 6 al salir de G1, ttl 5 al salir de sus vecinos directos, y ttl 3 y 2 las que ya fueron reenviadas tres y cuatro veces. La prueba se cumple.

6.2. Tipos de paquete (prueba P3)

Se esperaba que ICMP entregara menos que UDP, porque un eco necesita que lleguen la ida y la vuelta, y que TCP entregara más que UDP, a cambio de más transmisiones. La Tabla 6.1 lo confirma con 6 flujos entre usuarios en el despliegue de referencia.

UDP entrega 0.772 de los paquetes. ICMP entrega 0.646: el eco recorre el camino dos veces, y basta con que falle uno de los dos recorridos para que se pierda. TCP entrega 0.924, porque vuelve a enviar lo que no se confirma: los paquetes perdidos por el radio bajan de 0.155 a 0.007. El costo es claro: TCP usa 19.39 transmisiones por paquete, cerca de 2.6 veces las de UDP. Lo que TCP no arregla son los paquetes sin ruta en la tabla (0.041), porque reenviar no crea una ruta que el nodo no tiene. La Figura 5.7 muestra el mismo comportamiento en una sesión de la interfaz. La prueba se cumple.

UNIVERSIDAD
SERGIO ARBOLEDA

Cuadro 6.1: Entrega según el tipo de paquete en el despliegue de referencia

Tipo de paquete	**Entrega de datos**	**Transmisiones por paquete**	**Saltos**	**Perdidos por el radio**	**Perdidos sin ruta**
UDP (sin confirmación)	0.772 ± 0.061	7.46 ± 1.96	3.91 ± 0.74	0.155 ± 0.048	0.053 ± 0.035
ICMP (eco: ida y vuelta)	0.646 ± 0.082	11.83 ± 2.67	3.85 ± 0.72	0.256 ± 0.060	0.079 ± 0.024
TCP (confirmación y retransmisión)	0.924 ± 0.024	19.39 ± 4.46	3.97 ± 0.74	0.007 ± 0.007	0.041 ± 0.012

Media ± desviación estándar de 10 corridas de 240 s con 6 flujos entre Nodos de usuario y nodos fijos (lote `lotes/tipos_paquete.json`). Un eco de ICMP cuenta como entregado solo si la respuesta vuelve al origen; un segmento de TCP, si llegó al destino, con hasta 3 retransmisiones.

6.3. Fallos con Gateway fijos (prueba P4)

Esta prueba mide cómo responde la malla a cuatro tipos de fallo cuando los Gateway no se mueven. La Tabla 6.2 resume la respuesta y la Figura 6.1 muestra la cobertura a lo largo del tiempo.

Cuadro 6.2: Respuesta de la malla a cada fallo

Experimento	**Malla partida (s)**	**Reconvergencia de rutas (s)**	**Alertas**	**Cobertura media**	**Nodos cubiertos al final**
Control (sin fallos)	0.0	no aplica	3.1 ± 2.6	1.000	6
Gateway redundante	0.0	7.7 ± 3.7	16.8 ± 2.4	1.000	6
Gateway puente	90.0	19.4 ± 6.7	22.0 ± 1.8	1.000	6
Gateway de borde	0.0	17.1 ± 9.0	11.8 ± 1.7	0.938	6
Cascada	40.0	no aplica	20.1 ± 0.3	0.736	3

Media ± desviación estándar muestral de 10 corridas de 240 s simulados (semillas 1 a 10); sin ±, la desviación es 0. "No aplica": ningún episodio de reconvergencia se cerró.

Comparación con lo calculado a mano. Tres de los resultados esperados se pueden calcular a partir del escenario, sin simular:

- *Caída de borde:* N6, uno de 6 usuarios, queda sin cobertura durante 90 s de los 240 s. La cobertura media debe ser $1 - \frac{1}{6} \cdot \frac{90}{240} = 0{,}9375$, donde $\frac{1}{6}$ es la fracción de usuarios afectados y $\frac{90}{240}$ la fracción del tiempo. La simulación da 0.938.
- *Cascada:* hay 6 usuarios cubiertos durante 100 s, 4 durante 40 s y 3 durante 100 s. La cobertura media debe ser $\frac{6 \cdot 100 + 4 \cdot 40 + 3 \cdot 100}{6 \cdot 240} = 0{,}736$, es decir, los usuarios cubiertos en cada tramo por su duración, sobre el total de usuarios por el total de tiempo. La simulación da 0.736, con 3 usuarios cubiertos al final.

UNIVERSIDAD SERGIO ARBOLEDA

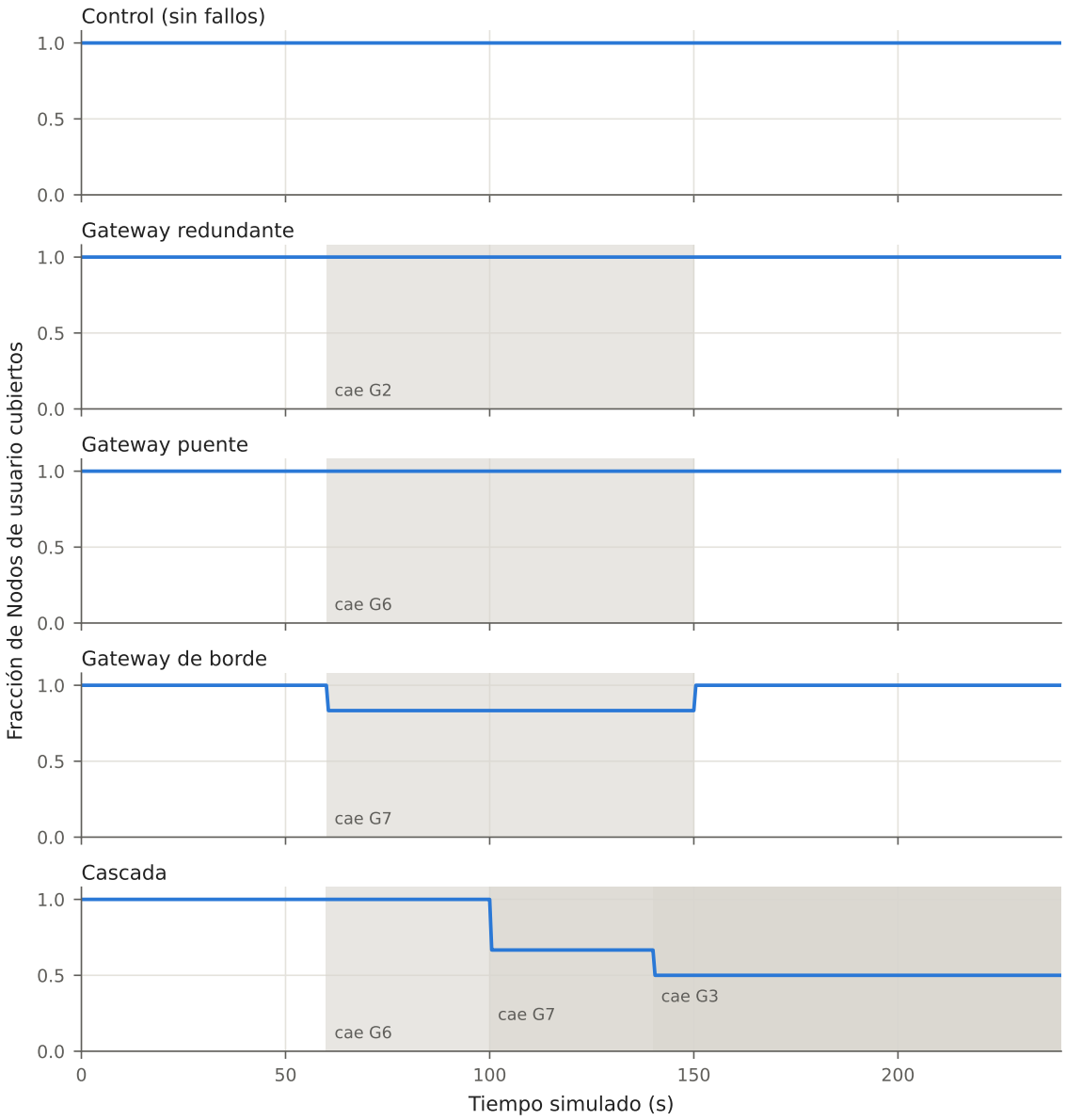


Figura 6.1: Cobertura a lo largo del tiempo en cada caso de la prueba P4. Cada panel es un caso: sin fallos, caída del Gateway redundante, del puente, del de borde y la cascada. La línea es el promedio de las 10 semillas y la banda de color, el rango entre la menor y la mayor. Las franjas grises marcan el tiempo en que hay algún Gateway caído. La cobertura solo baja en la caída de borde y en la cascada

UNIVERSIDAD
SERGIO ARBOLEDA

- *Puente:* la malla debe estar partida exactamente mientras G6 está caído, 90 s, sin diferencia entre semillas. La simulación da 90.0.

Los tres coinciden. Estas comparaciones validan la forma de contar de las métricas físicas; no dicen nada sobre el protocolo.

La cobertura no muestra la partición. Es el primer resultado importante: la caída del puente mantiene la malla partida los 90 s que dura y, sin embargo, la cobertura media es de 1.000, porque cada parte conserva algún Gateway. En cambio, la caída de borde no parte la malla y sí reduce la cobertura, a 0.938. Las dos métricas miden daños distintos, y un reporte basado en una sola de ellas daría una imagen falsa del fallo. La Figura 5.3 muestra precisamente ese estado.

Tiempo de adaptación de las rutas. Cuando vuelve un Gateway, las rutas tardan 7.7 ± 3.7 s en rehacerse si se cayó uno redundante, y 19.4 ± 6.7 s si se cayó el puente, porque en ese caso hay que restablecer rutas entre dos partes que estuvieron separadas (Figura 6.2). En la cascada no aplica, porque ningún nodo vuelve.

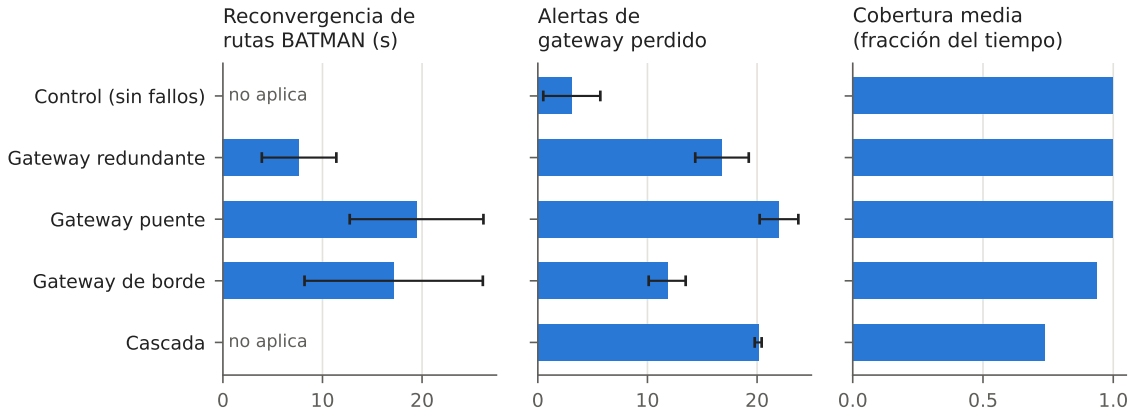


Figura 6.2: Comparación de los cinco casos de la prueba P4 en tres métricas, una por panel: tiempo que tardan las rutas en rehacerse, número de alertas de Gateway perdido y cobertura media. Cada barra es el promedio de 10 semillas y la línea negra, la desviación estándar. "No aplica" indica que en ese caso no hubo un episodio que medir

Detección y falsas alarmas. Los Gateway detectan las caídas sin coordinador, pero con ruido. La primera alerta llega a los 72.5 ± 9.2 s con la caída del puente y a los 78.0 ± 12.7 s con la del Gateway redundante, es decir, menos de 30 s después de la caída: el tiempo límite se cuenta desde el último mensaje oído, y en enlaces con pérdidas ya venía acumulado algo de silencio. El caso sin fallos muestra el costo de esa sensibilidad: hay 3.1 ± 2.6 alertas falsas. Con enlaces de fiabilidad entre 0.41 y 0.65, un tiempo límite de 30 s produce falsas alarmas de vez en cuando. Es un compromiso que la herramienta permite explorar cambiando un solo parámetro.

UNIVERSIDAD SERGIO ARBOLEDA

Calidad de las rutas. La Tabla 6.3 indica que el TQ medio es parecido en todos los casos, entre 0.32 y 0.36. Es un valor bajo porque el TQ se multiplica en cada salto, sobre enlaces de fiabilidad media y rutas de casi 3 saltos. Que suba un poco con los fallos más graves no es una mejora: al perderse nodos desaparecen del promedio las rutas más largas, que son las de peor TQ.

Cuadro 6.3: Calidad de las rutas y del medio en los experimentos de fallo

Experimento	**TQ medio de rutas**	**Saltos medios por ruta**	**Tasa de entrega del radio**
Control (sin fallos)	0.331 ± 0.005	2.82 ± 0.03	0.600 ± 0.002
Gateway redundante	0.323 ± 0.004	2.84 ± 0.02	0.606 ± 0.003
Gateway puente	0.339 ± 0.007	2.85 ± 0.03	0.603 ± 0.003
Gateway de borde	0.340 ± 0.007	2.81 ± 0.04	0.599 ± 0.002
Cascada	0.359 ± 0.010	2.87 ± 0.04	0.601 ± 0.003

6.4. Los mismos fallos con Gateway móviles (prueba P5)

Se esperaba que, si los Gateway pueden moverse, la malla pasara menos tiempo partida. Las Tablas 6.4 y 6.5 repiten los casos anteriores con el movimiento descentralizado.

Cuadro 6.4: Respuesta de la malla a cada fallo con Gateway móviles (descentralizada)

Experimento	**Malla partida (s)**	**Reconvergencia de rutas (s)**	**Alertas**	**Cobertura media**	**Nodos cubiertos al final**
Control (sin fallos)	0.0	no aplica	6.6 ± 5.6	1.000	6
Gateway redundante	0.0	15.5 ± 9.5	14.5 ± 2.9	1.000 ± 0.001	6
Gateway puente	6.2 ± 6.4	9.7 ± 7.0	14.3 ± 2.7	0.922 ± 0.043	6 ± 0
Gateway de borde	17.0 ± 53.8	30.1 ± 25.5	18.3 ± 7.7	0.959 ± 0.024	6
Cascada	6.2 ± 6.4	4.3 ± 4.0	17.2 ± 2.8	0.826 ± 0.054	4 ± 1

Media ± desviación estándar muestral de 10 corridas de 240 s simulados (semillas 1 a 10); sin ±, la desviación es 0. “No aplica”: ningún episodio de reconvergencia se cerró.

La malla se adapta sola, con costos. Sin ninguna orden central, los Gateway que quedan se reubican. La partición por la caída del puente baja de 90 s a 6.2 ± 6.4 s, y la cascada deja 4 ± 1 usuarios cubiertos y una cobertura de 0.826, frente a 3 y 0.736 con Gateway fijos. La prueba se cumple. Pero moverse también cuesta. En la caída del

UNIVERSIDAD SERGIO ARBOLEDA

Cuadro 6.5: Calidad de las rutas y del medio en los experimentos de fallo con Gateway móviles (descentralizada)

Experimento	**TQ medio de rutas**	**Saltos medios por ruta**	**Tasa de entrega del radio**
Control (sin fallos)	0.337 ± 0.010	2.82 ± 0.04	0.605 ± 0.006
Gateway redundante	0.348 ± 0.011	2.77 ± 0.03	0.603 ± 0.004
Gateway puente	0.361 ± 0.011	2.76 ± 0.05	0.597 ± 0.006
Gateway de borde	0.346 ± 0.010	2.81 ± 0.05	0.603 ± 0.006
Cascada	0.364 ± 0.009	2.76 ± 0.05	0.599 ± 0.007

puente la cobertura baja de 1.000 a 0.922 ± 0.043, y en la del Gateway redundante el tiempo de adaptación de las rutas se duplica, de 7.7 s a 15.5 ± 9.5 s, porque la red sigue cambiando mientras se rehacen las rutas. El caso sin fallos también tiene más alertas (6.6 frente a 3.1). No es una mejora en todo, sino un intercambio entre conectividad, cobertura y estabilidad.

Una observación sobre la dispersión: en la caída de borde, el tiempo con la malla partida es de 17.0 ± 53.8 s. Ese promedio lo produce una sola semilla entre diez; no conviene leerlo como el comportamiento típico.

6.5. Modos de movimiento (pruebas P6 y P7)

P6, sin tráfico. Se esperaba que el movimiento descentralizado mantuviera la malla unida con una cobertura cercana a la de los modos centralizados. La Tabla 6.6 y la Figura 6.3 comparan los tres modos.

Los resultados muestran el defecto de cada regla centralizada. Con *seguir*, los Gateway se amontonan: la entrega del radio es alta porque los enlaces son cortos, pero la cobertura es baja (0.40 en `base`, 0.51 en `denso`). Con *repartir* la cobertura es casi total, pero la malla pasa partida casi toda la corrida (178 s de 200 s en `base`). El movimiento descentralizado queda en un punto intermedio útil: en `base` y `denso` mantiene la malla casi siempre unida (5 s y 0 s partida) con cobertura de 0.84 y 0.99. No gana en todo: en `colapso_progresivo` su cobertura (0.42) es muy inferior a la de *repartir* (0.82). Y paga un precio: el TQ y la tasa de entrega del radio son los más bajos, porque la malla estirada usa enlaces más largos. La prueba se cumple en parte: claramente en `base` y `denso`; en los otros tres escenarios la malla pasa menos tiempo partida que con *repartir*, pero con menos cobertura.

P7, con tráfico. Se esperaba que el modo que conecta a más nodos entregara más paquetes. La Tabla 6.7 y la Figura 6.4 añaden 6 flujos de datos.

Según la Tabla 6.10, el movimiento descentralizado entrega más datos que *repartir* en los tres escenarios, y más que *seguir* en `base` y `denso`. En `colapso_progresivo`, la diferencia con *seguir* no es concluyente: el intervalo incluye el cero. Las entregas son bajas en todos los casos, entre 0.14 y 0.28. De dónde sale la ganancia también enseña

UNIVERSIDAD SERGIO ARBOLEDA

Cuadro 6.6: Movilidad de los Gateway: *seguir*, *repartir* y *descentralizada*

Escenario	**Movilidad**	**Cobertura media**	**Componentes al final**	**Malla partida (s)**	**Entrega del radio**	**TQ medio**
base	seguir	0.40 ± 0.19	1.1 ± 0.3	16 ± 52	0.86 ± 0.03	0.89 ± 0.03
base	repartir	0.95 ± 0.00	3.0	178 ± 1	0.49 ± 0.01	0.77 ± 0.03
base	descentralizada	0.84 ± 0.11	1.0	5 ± 15	0.27 ± 0.01	0.47 ± 0.04
colapso_progresivo	seguir	0.36 ± 0.07	1.0	5 ± 15	0.80 ± 0.02	0.86 ± 0.03
colapso_progresivo	repartir	0.82 ± 0.00	1.0	102 ± 1	0.47 ± 0.02	0.75 ± 0.03
colapso_progresivo	descentralizada	0.42 ± 0.06	1.0	0	0.27 ± 0.02	0.51 ± 0.05
particion	seguir	0.67	2.0	200	0.96 ± 0.01	0.98 ± 0.01
particion	repartir	0.99 ± 0.00	3.0	200	0.79 ± 0.02	0.88 ± 0.02
particion	descentralizada	0.77 ± 0.13	1.8 ± 0.4	195 ± 12	0.47 ± 0.04	0.55 ± 0.05
rescatista_perdido	seguir	0.91 ± 0.01	2.0	159 ± 0	0.74 ± 0.06	0.79 ± 0.03
rescatista_perdido	repartir	0.93 ± 0.00	2.0	159 ± 0	0.48 ± 0.01	0.70 ± 0.02
rescatista_perdido	descentralizada	0.75 ± 0.08	1.1 ± 0.3	59 ± 17	0.29 ± 0.02	0.56 ± 0.03
denso	seguir	0.51 ± 0.07	1.0	0	0.76 ± 0.03	0.82 ± 0.02
denso	repartir	0.99 ± 0.00	1.7 ± 0.5	85 ± 53	0.35 ± 0.02	0.50 ± 0.05
denso	descentralizada	0.99 ± 0.01	1.0	0	0.30 ± 0.00	0.38 ± 0.01

Media ± desviación estándar de 10 corridas de 200 s simulados (lote `lotes/movilidad.json`). *seguir* y *repartir* son la línea base centralizada; *descentralizada* decide cada Gateway solo con lo que oye.

Cuadro 6.7: Plano de datos según la movilidad de los Gateway

Escenario	**Movilidad**	**Entrega de datos**	**Entrega con camino**	**Pérdida sin camino**	**Estiramiento**	**Exactitud del conocimiento**
base	seguir	0.21 ± 0.13	0.40 ± 0.15	0.53 ± 0.28	1.04 ± 0.12	0.95 ± 0.02
base	repartir	0.20 ± 0.12	0.35 ± 0.13	0.47 ± 0.24	1.53 ± 0.17	0.90 ± 0.01
base	descentralizada	0.28 ± 0.15	0.30 ± 0.15	0.10 ± 0.05	1.03 ± 0.06	0.55 ± 0.05
colapso_progresivo	seguir	0.22 ± 0.13	0.41 ± 0.15	0.53 ± 0.28	1.01 ± 0.04	0.94 ± 0.03
colapso_progresivo	repartir	0.20 ± 0.12	0.33 ± 0.13	0.47 ± 0.24	1.67 ± 0.23	0.88 ± 0.01
colapso_progresivo	descentralizada	0.23 ± 0.14	0.33 ± 0.16	0.36 ± 0.19	1.00	0.71 ± 0.07
denso	seguir	0.14 ± 0.08	0.30 ± 0.14	0.54 ± 0.14	1.20 ± 0.26	0.89 ± 0.06
denso	repartir	0.18 ± 0.09	0.28 ± 0.08	0.38 ± 0.20	1.41 ± 0.18	0.82 ± 0.07
denso	descentralizada	0.28 ± 0.07	0.28 ± 0.07	0.01 ± 0.01	1.32 ± 0.25	0.86 ± 0.03

Media ± desviación estándar de 10 corridas de 200 s con 6 flujos de datos aleatorios (lote `lotes/movilidad_datos.json`).

UNIVERSIDAD
SERGIO ARBOLEDA

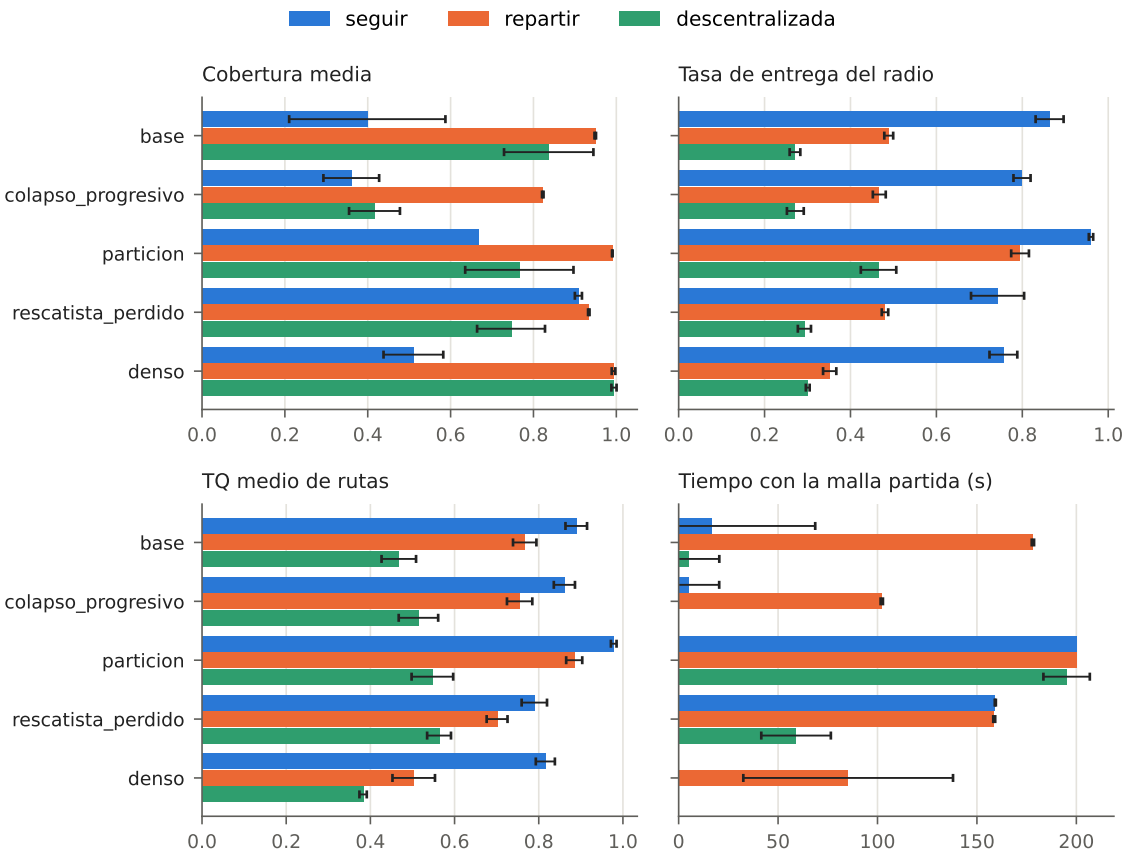


Figura 6.3: Comparación de los tres modos de movimiento de los Gateway (*seguir*, *repartir* y descentralizado) en los cinco escenarios de la prueba P6. Cada panel es una métrica: cobertura media, tasa de entrega del radio, TQ medio de las rutas y tiempo con la malla partida. Cada barra es el promedio de 10 semillas. Ningún modo es el mejor en las cuatro métricas a la vez

UNIVERSIDAD
SERGIO ARBOLEDA

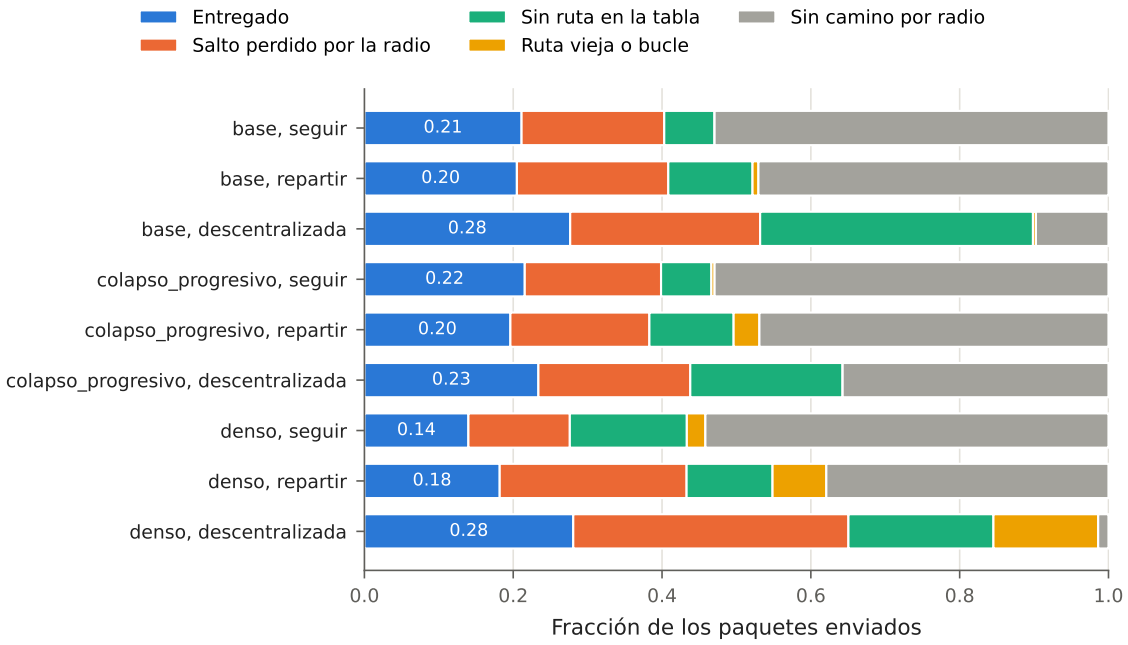


Figura 6.4: Qué pasó con los paquetes enviados en la prueba P7, según el escenario y el modo de movimiento. Cada barra representa todos los paquetes enviados (suma 1) y se divide en cinco partes: entregados, perdidos por el radio en algún salto, perdidos porque un nodo no tenía ruta, perdidos por una ruta vieja o un bucle, y sin ningún camino por radio. El modo descentralizado reduce sobre todo la parte gris (sin camino), pero aumenta las pérdidas por el radio y por falta de ruta

UNIVERSIDAD
SERGIO ARBOLEDA

algo: el movimiento descentralizado reduce sobre todo los paquetes sin ningún camino por radio, pero aumenta los perdidos sin ruta y por el radio. Entre los pares que sí tienen camino, entrega menos que *seguir* en `base` (−0,104) y sin diferencia concluyente en los otros dos. Gana porque conecta a más pares, no porque lleve mejor los paquetes. La prueba se cumple.

Las tablas de rutas se quedan atrás. Este es el costo más revelador, y solo se ve con las métricas de lo que cree cada nodo. En `base`, la exactitud del conocimiento baja de 0.95 con *seguir* a 0.55 con el movimiento descentralizado: en un 44 % de los pares, el radio los conecta pero el nodo no tiene una ruta vigente. La malla que el movimiento logra mantener unida es tan débil que los OGM no alcanzan a mantener las rutas. Una métrica de conectividad física diría que la red está mejor, cuando lo que hay es una red conectada que no se conoce a sí misma. Esto sugiere ajustar el movimiento para que tenga en cuenta la calidad de los enlaces y no solo el alcance.

6.6. Fallos al azar y fallos dirigidos (prueba P8)

Se esperaba que dos caídas dirigidas al nodo más crítico hicieran más daño que dos caídas al azar. La Tabla 6.8 las compara.

Cuadro 6.8: Dos caídas al azar contra dos caídas dirigidas al nodo más crítico

Experimento	**Malla partida (s)**	**Entrega de datos**	**Entrega con camino**	**Cobertura media**	**Rutas fantasma**
Fallos al azar	42 ± 57	0.502 ± 0.114	0.686 ± 0.069	0.925 ± 0.067	0.020 ± 0.013
Fallos dirigidos	180	0.284 ± 0.088	0.696 ± 0.087	0.917	0.052 ± 0.003

Media ± desviación estándar de 10 corridas de 240 s con 6 flujos de datos, caídas a los 60 y 120 s (lote `lotes/robustez.json`).

Los fallos dirigidos parten la malla durante 180 s en todas las semillas, frente a 42 ± 57 s al azar, y dejan la entrega de datos en 0.284 frente a 0.502, una diferencia pareada de 0.218 (IC 95 %: 0.149 a 0.287). La prueba se cumple. Recuerda la distinción clásica entre errores y ataques [Albert et al., 2000], aunque ese resultado se refiere a redes de otro tipo y esta es una malla pequeña, de modo que es una analogía y no una confirmación. Lo importante es lo que *no* cambia: la cobertura media (0.925 y 0.917) no distingue los dos casos, con un intervalo que incluye el cero, porque cada parte conserva algún Gateway. Tampoco cambia la entrega entre pares con camino (0.686 y 0.696): el enrutamiento sigue funcionando dentro de cada parte, y el daño es la falta de camino, que explica el 59.9 % de los paquetes en los fallos dirigidos. Sin paquetes de datos, este daño no se habría visto con la métrica de cobertura.

UNIVERSIDAD
SERGIO ARBOLEDA

6.7. B.A.T.M.A.N. e inundación (prueba P9)

Se esperaba que la inundación entregara más que B.A.T.M.A.N. a cambio de más transmisiones. La Tabla 6.9 y la Figura 6.5 comparan los dos protocolos en dos mapas.

Cuadro 6.9: B.A.T.M.A.N. contra inundación en dos mapas

Entorno	**Protocolo**	**Entrega de datos**	**Entrega con camino**	**Transmisiones por paquete**	**Saltos**
Campo abierto, usuarios en grupos	B.A.T.M.A.N.	0.575 ± 0.128	0.803 ± 0.090	2.86 ± 0.91	1.55 ± 0.21
Campo abierto, usuarios en grupos	Inundación	0.607 ± 0.127	0.937 ± 0.042	5.62 ± 1.38	1.56 ± 0.20
Campo abierto, random waypoint	B.A.T.M.A.N.	0.446 ± 0.064	0.544 ± 0.078	3.38 ± 1.08	1.79 ± 0.23
Campo abierto, random waypoint	Inundación	0.618 ± 0.063	0.743 ± 0.071	7.40 ± 1.27	2.24 ± 0.29
Edificio, nodos fijos	B.A.T.M.A.N.	0.786 ± 0.054	0.786 ± 0.054	7.19 ± 1.73	3.90 ± 0.74
Edificio, nodos fijos	Inundación	0.430 ± 0.081	0.430 ± 0.081	6.96 ± 0.34	4.30 ± 0.78

Media ± desviación estándar de 10 corridas de 300 s (lote `lotes/generalidad.json`). B.A.T.M.A.N. reintenta cada salto hasta 3 veces (unicast); la inundación difunde una vez, sin reintentos.

La respuesta depende del lugar, y esta es una razón para no atar un simulador a un solo mapa. En campo abierto se cumple lo esperado: la inundación entrega más que B.A.T.M.A.N. entre pares con camino (0.937 frente a 0.803 con usuarios en grupos; 0.743 frente a 0.544 con usuarios que caminan al azar), con cerca del doble de transmisiones por paquete. Cuando los usuarios se mueven al azar, la causa principal de pérdida de B.A.T.M.A.N. es la ruta vieja (27.6 % de los paquetes): las tablas no siguen el ritmo del movimiento. En el edificio ocurre lo contrario y la prueba no se cumple: B.A.T.M.A.N. entrega 0.786 frente a 0.430 de la inundación, con un número de transmisiones parecido (7.19 y 6.96). Allí las rutas son largas (casi 4 saltos) sobre enlaces de fiabilidad media; B.A.T.M.A.N. reintenta cada salto hasta tres veces y la inundación no, por lo que sus copias se van perdiendo por el camino.

Hay dos precauciones. Primero, las transmisiones por paquete solo cuentan los datos, no los OGM que B.A.T.M.A.N. necesita y la inundación no. Segundo, el medio no representa colisiones, de modo que no refleja el principal costo conocido de la inundación, que es saturar el medio [Ni et al., 1999]. Los dos sesgos van en sentidos contrarios, y la comparación vale para la entrega dentro de este modelo, no como un veredicto sobre los protocolos.

La Tabla 6.10 reúne las diferencias pareadas de las pruebas P7, P8 y P9.

UNIVERSIDAD SERGIO ARBOLEDA

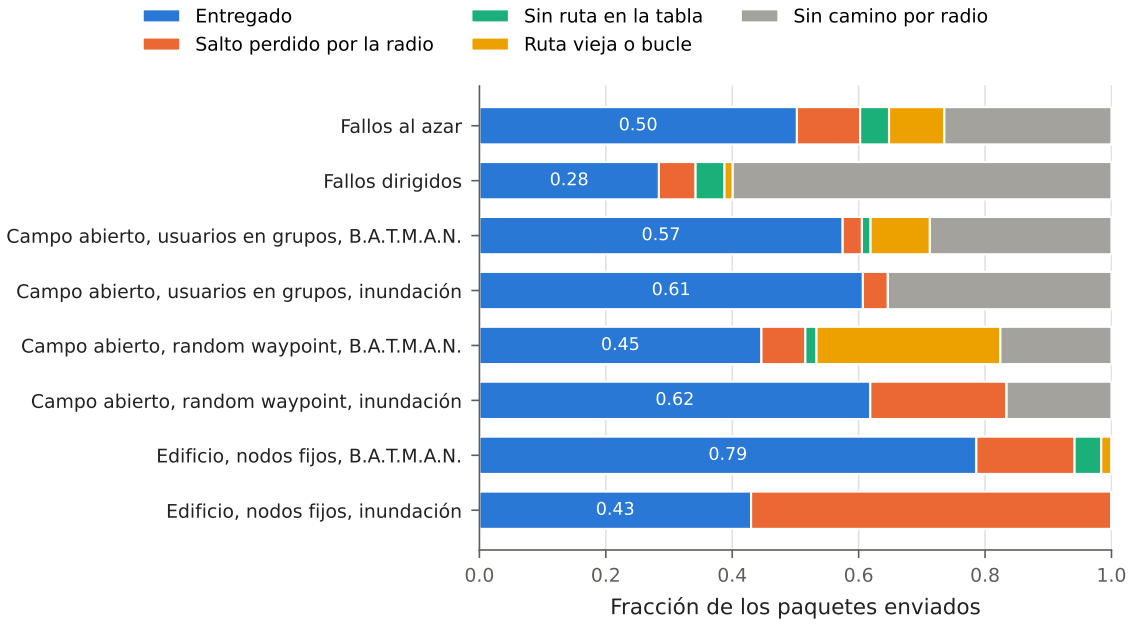


Figura 6.5: Qué pasó con los paquetes enviados en las pruebas P8 (fallos al azar y dirigidos) y P9 (B.A.T.M.A.N. e inundación en campo abierto y en el edificio). Cada barra representa todos los paquetes enviados y se divide en las mismas cinco partes de la Figura 6.4. En los fallos dirigidos domina la parte gris (sin camino por radio); en el edificio, la inundación pierde la mayoría de los paquetes por el radio

UNIVERSIDAD
SERGIO ARBOLEDA

Cuadro 6.10: Diferencias pareadas por semilla, con intervalo de confianza del 95 %

Comparación	**Diferencia media**	**Intervalo del 95 %**	**¿Excluye el 0?**
base: entrega, descentralizada − seguir	+0.065	[+0.020, +0.111]	sí
base: entrega, descentralizada − repartir	+0.072	[+0.032, +0.111]	sí
base: entrega con camino, descentralizada − seguir	−0.104	[−0.195, −0.014]	sí
colapso_progresivo: entrega, descentralizada − seguir	+0.018	[−0.016, +0.052]	no
colapso_progresivo: entrega, descentralizada − repartir	+0.038	[+0.018, +0.058]	sí
colapso_progresivo: entrega con camino, descentralizada − seguir	−0.080	[−0.166, +0.005]	no
denso: entrega, descentralizada − seguir	+0.141	[+0.106, +0.176]	sí
denso: entrega, descentralizada − repartir	+0.099	[+0.060, +0.138]	sí
denso: entrega con camino, descentralizada − seguir	−0.012	[−0.083, +0.060]	no
Robustez: entrega, azar − dirigida	+0.218	[+0.149, +0.287]	sí
Robustez: cobertura, azar − dirigida	+0.008	[−0.040, +0.057]	no
Campo abierto, usuarios en grupos: entrega con camino, inundación − B.A.T.M.A.N.	+0.134	[+0.067, +0.202]	sí
Campo abierto, random waypoint: entrega con camino, inundación − B.A.T.M.A.N.	+0.199	[+0.114, +0.284]	sí
Edificio, nodos fijos: entrega con camino, inundación − B.A.T.M.A.N.	−0.356	[−0.388, −0.325]	sí

Intervalo t de Student con 9 grados de libertad sobre las diferencias entre corridas con la misma semilla. Sin corrección por comparaciones múltiples.

UNIVERSIDAD
SERGIO ARBOLEDA

6.8. Repetibilidad y tamaño de la red (pruebas P10 y P11)

P10, repetibilidad. Se esperaba que la misma semilla diera la misma corrida. El 4 de noviembre de 2026 se volvieron a correr todos los lotes en un computador distinto de aquel en el que se habían generado las tablas el 27 de octubre, y las cinco tablas de esos lotes salieron idénticas byte a byte. El 13 de noviembre, después de añadir los tipos de paquete, se repitieron los tres lotes con tráfico de datos y sus tablas volvieron a salir idénticas. También se comprobó que la interfaz web produce la misma serie de métricas que la terminal con la misma semilla. La prueba se cumple: los resultados no solo son parecidos, son exactamente los mismos (requerimiento R6).

P11, tamaño de la red. La Tabla 5.4 muestra que el simulador corre más rápido que el tiempo real hasta unas decenas de nodos y más lento a partir de unos 100 nodos juntos. Ese es su rango de uso.

6.9. Lo que las pruebas mostraron sobre el protocolo

Llevar los paquetes con las tablas de cada nodo dejó ver dos comportamientos que antes no se podían observar:

- **Bucles.** Entre el 1.3 % y el 2.3 % de los paquetes se pierde dando vueltas entre nodos, en las pruebas P8 y P9 con B.A.T.M.A.N. La causa probable es que esta implementación reenvía la primera copia de cada OGM, en lugar de solo la del mejor vecino, como hace el protocolo.
- **Reparación lenta.** Cuando se cae un Gateway, la ruta que pasaba por él permanece en las tablas cerca de 50 s (observado al examinar corridas de la caída del Gateway redundante), porque solo se reemplaza cuando aparece otra mejor. En la prueba de fallos al azar, el 7.4 % de los paquetes se pierde por este motivo. El protocolo borra al nodo de origen cuando pasa un tiempo sin oírlo.
- **Efecto de corregir el TQ.** Antes de la corrección, el TQ de toda ruta valía 1.0, de modo que la medida central del protocolo no decía nada. Con nodos fijos y la semilla 1, el TQ medio pasó a 0.740 en `base`, 0.514 en `denso` y 0.711 en `particion`. La red parecía perfecta sin serlo.

Los dos primeros no se corrigieron, a propósito: hacerlo cambiaría el funcionamiento del protocolo, y se prefirió reportarlos. Son el tipo de hallazgo que justifica ejecutar el código en lugar de un modelo idealizado: una copia escrita con la especificación en mente no los habría producido. Y, a la inversa, marcan el límite de los resultados: parte de las pérdidas de este capítulo pertenecen a esta implementación y no a B.A.T.M.A.N. en general.

UNIVERSIDAD
SERGIO ARBOLEDA

6.10. Resumen de las pruebas

Cuadro 6.11: Resumen de las pruebas: lo que se esperaba, lo que se obtuvo y si coinciden

N.º	**Qué se esperaba**	**Qué se obtuvo**	**¿Coincide?**
P1	Casi todos los pares conectados con ruta a los 30 s	152 de 156 pares (97 %)	Sí
P2	El TTL baja en uno por salto	De 6 al salir del origen a 2 tras cuatro reenvíos	Sí
P3	ICMP < UDP < TCP en entrega; TCP con más transmisiones	0.646, 0.772 y 0.924; TCP usa 2.6 veces las transmisiones de UDP	Sí
P4	Partición de 90 s por el puente; cobertura de 0.9375 por el borde y de 0.736 en la cascada	90.0 s; 0.938; 0.736	Sí
P5	Menos tiempo con la malla partida	De 90 s a 6.2 s en la caída del puente, con cobertura de 0.922	Sí, con costo
P6	Malla unida con cobertura cercana a la centralizada	Se logra en `base` y `denso`; en los otros tres escenarios la cobertura es menor que con *repartir*	En parte
P7	Más entrega al conectar a más nodos	Más entrega que *repartir* en 3 de 3 escenarios y que *seguir* en 2 de 3	Sí
P8	Los fallos dirigidos hacen más daño	Entrega de 0.284 frente a 0.502; la cobertura no los distingue	Sí
P9	La inundación entrega más, con más transmisiones	Sí en campo abierto; en el edificio entrega menos (0.430 frente a 0.786)	En parte
P10	Tablas idénticas al repetir	Idénticas byte a byte en dos computadores	Sí
P11	El costo crece con los nodos	Más rápido que el tiempo real hasta decenas de nodos	Sí

6.11. Discusión

La pregunta del planteamiento era cómo implementar un simulador para redes descentralizadas que ejecute el código de los nodos, represente el medio de un edificio, decida solo con información local y permita medir la cobertura y la adaptabilidad. Las pruebas permiten responderla por partes.

Qué requerimientos fueron decisivos. Separar las pérdidas físicas de las de enrutamiento y medir lo que cree cada nodo frente a la realidad (R5) explica buena parte de los hallazgos de este capítulo. La regla de decidir solo con información local (R4) no es un capricho: al aplicarla aparecieron las tablas de rutas atrasadas y los

UNIVERSIDAD SERGIO ARBOLEDA

bucles, ocultos mientras los mensajes se entregaban por la red real. En cambio, R7 se cumple solo en parte, como muestra la Tabla 5.4.

Cobertura. El resultado que más se repite es que *una sola medida de conectividad oculta el daño*: la cobertura no distingue una partición, no separa un fallo dirigido de uno al azar, y la conectividad física no refleja que los nodos desconocen sus rutas. Para evaluar una red descentralizada hay que medir a la vez la física, la entrega de datos y lo que cree cada nodo.

Adaptabilidad. El movimiento descentralizado consigue resultados que compiten con las reglas centralizadas sin usar información global, y permite que la malla se adapte sola a un fallo. Sus costos son concretos: enlaces más débiles, tablas de rutas atrasadas y más alertas. Esto es coherente con el compromiso entre cobertura y conectividad que describe la literatura de despliegue autónomo [Poduri and Sukhatme, 2004].

Qué le falta al simulador frente a otras herramientas. La Tabla 6.12 compara el simulador con las herramientas revisadas en el Capítulo 3. Lo que este trabajo aporta está en las últimas filas; lo que le falta, en las primeras, y coincide con las limitaciones declaradas.

Hasta dónde llegan estas conclusiones. Sin una validación con equipos físicos ni contra batman-adv, las magnitudes absolutas no se pueden trasladar a una red real. El medio es optimista en varios aspectos [Kotz et al., 2004] y los umbrales del movimiento se ajustaron sobre parte de los mismos escenarios. Los resultados más sólidos son los que tienen una causa física clara, como el de la cobertura frente a la partición; los más frágiles son las comparaciones de magnitud pequeña, sobre todo las diferencias con intervalos cercanos al cero.

UNIVERSIDAD
SERGIO ARBOLEDA

Cuadro 6.12: Comparación del simulador con las herramientas revisadas: qué ofrece cada una. Las primeras filas son lo que le falta a este simulador; las últimas, lo que aporta

Característica	**ns-3 y OMNeT++/INET**	**CORE, EMANE y Mininet-WiFi**	**Este simulador**
Colisiones, interferencia y acceso al medio	Sí, con modelos detallados [The ns-3 Project, 2026, INET Framework Project, 2026]	Sí, con EMANE [Adjacent Link LLC, 2026]	No
Modelos de propagación calibrados	Varios [The ns-3 Project, 2026]	Varios [Adjacent Link LLC, 2026]	Uno, simple y sin calibrar
Retardo de los paquetes	Sí	Sí, en tiempo real	No: cada salto ocurre dentro de un paso
Pila de Internet completa (IP, TCP con congestión)	Sí [INET Framework Project, 2026]	Sí, la de Linux [Ahrenholz et al., 2008, Fontes et al., 2015]	No: tres tipos de paquete simplificados
Redes de cientos de nodos	Sí	Más de cien [Ahrenholz et al., 2008]	Decenas
Ejecutar el código del nodo sin reescribirlo	Solo con *Direct Code Execution* [Tazaki et al., 2013]	Sí	Sí, las clases en Python del nodo
Modelo de pérdida entre pisos	ns-3 trae un módulo de edificios [The ns-3 Project, 2026]; hay que configurarlo	Hay que configurarlo	Sí, listo para usar
Medir lo que cree cada nodo frente a la realidad	Lo programa el usuario	Lo programa el usuario	Sí, incluido
Repetir exactamente una corrida	Sí, con semilla	Difícil: depende del tiempo real	Sí, con semilla

UNIVERSIDAD SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Capítulo 7

Conclusiones y recomendaciones

7.1. Conclusiones

Este trabajo implementó un simulador para redes descentralizadas y lo probó con simulaciones de una Red de Expansión de Cobertura. Las conclusiones siguen el orden de los objetivos específicos.

Sobre los requerimientos (objetivo 1). La revisión mostró que los simuladores de propósito general evalúan una copia del protocolo, que los emuladores ejecutan el programa real pero no traen un modelo listo de la pérdida de señal entre pisos, y que las herramientas de redes entre pares no representan el medio de radiofrecuencia. Además, cualquier simulador conoce toda la red, lo que facilita decisiones que ningún nodo real podría tomar; el simulador de partida lo hacía en tres lugares. De ahí salieron los diez requerimientos de la Tabla 5.1, entre ellos ejecutar el código del nodo, decidir solo con información local y medir lo que cree cada nodo.

Sobre el diseño (objetivo 2). La arquitectura en capas deja el código del nodo sin cambios y reemplaza únicamente la red por un modelo del medio que depende de la distancia y de los pisos. Los paquetes avanzan con la tabla de rutas de cada nodo y los Gateway se mueven solo con lo que oyen, con lo que se eliminaron los tres usos de información global, que quedan solo como referencia de comparación.

Sobre la implementación (objetivo 3). Se obtuvo un simulador que se configura con archivos, con dos tipos de mapa, tres modos de movimiento de los Gateway, tres de los usuarios, tres tipos de paquete (UDP, ICMP y TCP), caídas programables y cuatro formas de ejecución, entre ellas una interfaz web en la que cada función se puede observar. Su límite es el tamaño: corre más rápido que el tiempo real hasta unas decenas de nodos, y más lento con 100 nodos juntos. Su ventaja frente a la emulación es el control y las mediciones, no el tamaño.

Sobre las pruebas (objetivo 4). De las once pruebas de simulación, nueve dieron lo esperado y dos lo dieron en parte (Tabla 6.11). Con el despliegue de referencia y 10 semillas se concluye que:

- *Cobertura.* La cobertura no muestra el daño a la conectividad: la caída de un puente parte la malla 90 s con cobertura media de 1.000, y dos fallos dirigidos reducen la entrega de datos de 0.502 a 0.284 sin una diferencia de cobertura concluyente. Evaluar una red de este tipo exige medir a la vez la física, la entrega de datos y lo que cree cada nodo.
- *Adaptabilidad.* La malla se puede adaptar sin coordinador: con Gateway móviles,

UNIVERSIDAD
SERGIO ARBOLEDA

la partición por la caída del puente baja de 90 s a 6.2 s. El costo es una cobertura de 0.922 en ese mismo fallo, rutas que tardan más en rehacerse tras la caída de un nodo redundante y tablas de rutas menos exactas: en el escenario `base`, la exactitud del conocimiento baja de 0.95 a 0.55.

- *Tipos de paquete.* Con el mismo tráfico, TCP entrega 0.924, UDP 0.772 e ICMP 0.646; TCP lo logra con cerca de 2.6 veces las transmisiones de UDP.
- *Protocolos.* La comparación depende del lugar: la inundación supera a B.A.T.M.A.N. en campo abierto con cerca del doble de transmisiones, y queda muy por debajo en el edificio (0.430 frente a 0.786). Este resultado vale para un medio sin colisiones.
- *Repetibilidad.* Las tablas de resultados salieron idénticas byte a byte al repetir las corridas en dos computadores.

En conjunto, varios de estos resultados solo aparecieron al ejecutar el código del nodo, exigir decisiones locales y medir lo que cree cada nodo, y algunos contradijeron lo que el propio proyecto daba por cierto. Las conclusiones valen para esta implementación bajo este medio. No se hizo una validación con equipos físicos ni contra batman-adv, que es la principal tarea pendiente.

7.2. Recomendaciones

- **Validación externa.** Es la prioridad. Instalar el nodo de red (`mesh/node.py`) en un grupo pequeño de equipos de radio, repetir las pruebas de fallo y comparar con la simulación; y comparar una topología con batman-adv en un emulador o con un modelo en ns-3.
- **Acercar el protocolo a B.A.T.M.A.N.** Reenviar solo la copia del mejor vecino y borrar los nodos de origen que llevan tiempo en silencio, y medir cuánto se reducen los bucles y las pérdidas por rutas viejas.
- **Mejorar el medio.** Añadir enlaces que no sean simétricos, cambios en el tiempo, colisiones y una capa de acceso al medio, para responder a las críticas conocidas a los modelos simples [Kotz et al., 2004] y para evaluar la inundación con su costo real [Ni et al., 1999]. Calibrar el medio con mediciones en un edificio [Seidel and Rappaport, 1992].
- **Retardo y transporte.** Resolver los saltos con un retardo en lugar de dentro de un paso, lo que permitiría medir el retardo de los paquetes y representar TCP con sus tiempos de espera y su control de congestión [Eddy, 2022].
- **Movimiento que tenga en cuenta la calidad.** Incluir en las reglas la calidad de los enlaces y no solo el alcance, para que las tablas de rutas no se queden atrás. Evaluar los umbrales en escenarios que no se usaron para ajustarlos.

UNIVERSIDAD
SERGIO ARBOLEDA

- **Aprendizaje de las reglas locales.** El simulador ofrece un entorno repetible, con agentes que solo tienen información parcial, resultados medibles y un costo bajo por corrida. Esto permite plantear, como continuación en inteligencia artificial, que los Gateway aprendan su regla de movimiento, y compararla con la de este trabajo.
- **Tamaño.** Optimizar el procesamiento de los OGM, que consume la mayor parte del tiempo, para superar el límite actual de unos 100 nodos juntos.
- **Estadística.** Aumentar el número de semillas en las comparaciones de magnitud pequeña y aplicar una corrección por hacer varias comparaciones [Jain, 1991].

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Anexos

Anexo A. Reproducción de los resultados

Todas las tablas y figuras del Capítulo 6 se generan a partir de los lotes del repositorio. Se requiere Python 3 con las dependencias de `requirements.txt`.

Preparar el entorno

```
python3 -m venv venv
venv/bin/pip install -r requirements.txt
```

Correr las pruebas de simulación

Cada lote deja su salida en `reportes/`: el resumen estadístico, una fila por corrida y los reportes de cada semilla. Entre paréntesis, la prueba de la Tabla 5.5 a la que corresponde.

```
venv/bin/python main.py --batch lotes/tipos_paquete.json           # P3
venv/bin/python main.py --batch lotes/capitulo5_fallos.json        # P4
venv/bin/python main.py --batch lotes/capitulo5_fallos_movil.json  # P5
venv/bin/python main.py --batch lotes/movilidad.json               # P6
venv/bin/python main.py --batch lotes/movilidad_datos.json         # P7
venv/bin/python main.py --batch lotes/robustez.json                # P8
venv/bin/python main.py --batch lotes/generalidad.json             # P9
```

Generar tablas y figuras

La siguiente orden toma el lote más reciente de cada tipo y vuelve a escribir las tablas de `Plantilla/MainMatter/Cap5/tablas/` y las figuras de `Plantilla/Images/Cap5/`. No corre simulaciones.

```
venv/bin/python -m analysis.capitulo5
```

Observar una prueba en la interfaz

```
# Interfaz web local, con el escenario de la caída del puente (P1, P2 y P4)
venv/bin/python main.py --web --seed 1 \
    --config escenarios/fallos/fallo_puente.txt
```

UNIVERSIDAD
SERGIO ARBOLEDA

```
# La misma corrida sin ventana, con el resumen en la terminal
venv/bin/python main.py --headless --duracion 240 --seed 1 \
    --config escenarios/fallos/fallo_puente.txt
```

En la interfaz, la pestaña "Tráfico" permite elegir el tipo de paquete, la cantidad y el intervalo; la pestaña "Red" muestra la matriz de conocimiento, y la tecla O muestra los OGM del nodo seleccionado.

Compilar este documento

```
cd Plantilla
pdflatex main && bibtex main && pdflatex main && pdflatex main
```

El repositorio incluye una guía más amplia, con casos de uso y sus salidas esperadas, en `docs/guia_ejecucion.md`.

Anexo B. Comprobaciones automáticas del código

Además de las pruebas de simulación, el repositorio incluye 390 comprobaciones automáticas escritas con el módulo `unittest` de Python [Python Software Foundation, 2026]. Cada una ejecuta una parte pequeña del programa y compara el resultado con el valor correcto; por ejemplo, que un OGM perdido baje el TQ, que un enlace sin pérdidas lo mantenga en 1.0, o que una caída programada dé lo mismo que la misma caída hecha a mano en la interfaz. Sirven para detectar si un cambio en el programa dañó algo que ya funcionaba. La Tabla 7.1 muestra cómo se reparten. Se ejecutan con:

```
venv/bin/python -m unittest discover -s tests
```

Las 390 pasan; en Linux se omite una, que necesita un componente de macOS.

UNIVERSIDAD
SERGIO ARBOLEDA

Cuadro 7.1: Comprobaciones automáticas del código, por área del simulador

Área	**Comprobaciones**
Protocolo (TQ, elección de rutas) y medio de radiofrecuencia	17
Motor y nodos simulados	57
Lectura y revisión de escenarios	26
Fallos programables y adaptación de rutas	30
Envío de paquetes de datos y tipos de paquete	38
Movimiento descentralizado	13
Métricas de lo que cree cada nodo	12
Otros mapas, movimiento de usuarios e inundación	14
Reportes, lotes, uso por terminal y análisis	82
Interfaz web	101
Total	**390**

UNIVERSIDAD
SERGIO ARBOLEDA

UNIVERSIDAD
SERGIO ARBOLEDA

Bibliografía

[Adjacent Link LLC, 2026] Adjacent Link LLC (2026). EMANE: Extendable mobile ad-hoc network emulator. https://github.com/adjacentlink/emane. Documentación del proyecto. Consultado en noviembre de 2026. 28, 30, 77

[Ahrenholz et al., 2008] Ahrenholz, J., Danilov, C., Henderson, T. R., and Kim, J. H. (2008). CORE: A real-time network emulator. In *Proceedings of the IEEE Military Communications Conference (MILCOM 2008)*. IEEE. 23, 29, 30, 57, 77

[Akyildiz et al., 2005] Akyildiz, I. F., Wang, X., and Wang, W. (2005). Wireless mesh networks: a survey. *Computer Networks*, 47(4):445–487. 21, 24, 28, 32

[Albert et al., 2000] Albert, R., Jeong, H., and Barabási, A.-L. (2000). Error and attack tolerance of complex networks. *Nature*, 406(6794):378–382. 32, 48, 70

[Alharby and van Moorsel, 2020] Alharby, M. and van Moorsel, A. (2020). BlockSim: An extensible simulation tool for blockchain systems. *Frontiers in Blockchain*, 3. 29

[Aoki et al., 2019] Aoki, Y., Otsuki, K., Kaneko, T., Banno, R., and Shudo, K. (2019). SimBlock: A blockchain network simulator. In *IEEE INFOCOM 2019 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pages 325–329. IEEE. Preprint disponible en https://arxiv.org/abs/1901.09777. 29

[Baumgart et al., 2009] Baumgart, I., Heep, B., and Krause, S. (2009). OverSim: A scalable and flexible overlay framework for simulation and real network applications. In *Proceedings of the 9th IEEE International Conference on Peer-to-Peer Computing (P2P 2009)*, pages 87–88, Seattle, WA, EE.UU. IEEE. 29

[Breslau et al., 2000] Breslau, L., Estrin, D., Fall, K., Floyd, S., Heidemann, J., Helmy, A., et al. (2000). Advances in network simulation. *Computer*, 33(5):59–67. 27

[Camp et al., 2002] Camp, T., Boleng, J., and Davies, V. (2002). A survey of mobility models for ad hoc network research. *Wireless Communications and Mobile Computing*, 2(5):483–502. 32, 48

[Cavin et al., 2002] Cavin, D., Sasson, Y., and Schiper, A. (2002). On the accuracy of MANET simulators. In *Proceedings of the Second ACM International Workshop on Principles of Mobile Computing*, pages 38–43. 21, 24, 31

UNIVERSIDAD SERGIO ARBOLEDA

[Chroboczek and Schinazi, 2021] Chroboczek, J. and Schinazi, D. (2021). The Babel routing protocol. RFC 8966, RFC Editor. 31

[Clausen and Jacquet, 2003] Clausen, T. and Jacquet, P. (2003). Optimized link state routing protocol (OLSR). RFC 3626, Internet Engineering Task Force. 28, 31

[Corson and Macker, 1999] Corson, S. and Macker, J. (1999). Mobile ad hoc networking (MANET): Routing protocol performance issues and evaluation considerations. RFC 2501, RFC Editor. 21, 23, 28

[Diniz et al., 2025] Diniz, B. A., Ferrão, I. G., da Silva, L. M., and Branco, K. R. L. J. C. (2025). Comparative performance analysis of OLSR, BATMAN-Adv, and Babel in UAV mesh networks. In *2025 International Conference on Unmanned Aircraft Systems*, pages 496–503. 31

[Eddy, 2022] Eddy, W. (2022). Transmission control protocol (TCP). RFC 9293, RFC Editor. 47, 80

[Fang et al., 2021] Fang, Y., Cheng, B., Kang, K., and Tan, H. (2021). A self-adaptive deployment model of UAV cluster for emergency communication network. *International Journal of Distributed Sensor Networks*, 17(10). 32

[Fontes et al., 2015] Fontes, R. R., Afzal, S., Brito, S. H. B., Santos, M. A. S., and Rothenberg, C. E. (2015). Mininet-WiFi: Emulating software-defined wireless networks. In *Proceedings of the 11th International Conference on Network and Service Management (CNSM 2015)*, pages 384–389. IEEE. 23, 29, 77

[García et al., 2005] García, P., Pairot, C., Mondéjar, R., Pujol, J., Tejedor, H., and Rallo, R. (2005). PlanetSim: A new overlay network simulation framework. In *Software Engineering and Middleware*, Lecture Notes in Computer Science, pages 123–136. Springer. 29

[Harris et al., 2020] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., et al. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362. 48

[Hong et al., 1999] Hong, X., Gerla, M., Pei, G., and Chiang, C.-C. (1999). A group mobility model for ad hoc wireless networks. In *Proceedings of the 2nd ACM International Workshop on Modeling, Analysis and Simulation of Wireless and Mobile Systems*, pages 53–60. 32, 48

[Hopcroft and Tarjan, 1973] Hopcroft, J. and Tarjan, R. (1973). Algorithm 447: Efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6):372–378. 32, 48

[Howard et al., 2002] Howard, A., Matarić, M. J., and Sukhatme, G. S. (2002). Mobile sensor network deployment using potential fields: A distributed, scalable solution to

SERGIO ARBOLEDA

the area coverage problem. In *Distributed Autonomous Robotic Systems 5*, pages 299–308. Springer, Tokyo. 24, 32, 47

[Hunter, 2007] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95. 49

[IEEE, 2021] IEEE (2021). IEEE standard for information technology – telecommunications and information exchange between systems – local and metropolitan area networks – specific requirements – part 11: Wireless LAN medium access control (MAC) and physical layer (PHY) specifications. IEEE Std 802.11-2020. 46

[INET Framework Project, 2026] INET Framework Project (2026). INET framework. https://inet.omnetpp.org/. Documentación oficial. Consultado en noviembre de 2026. 27, 28, 77

[Jain, 1991] Jain, R. (1991). *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. John Wiley & Sons, New York. 39, 81

[Kiess and Mauve, 2007] Kiess, W. and Mauve, M. (2007). A survey on real-world implementations of mobile ad-hoc networks. *Ad Hoc Networks*, 5(3):324–339. 21, 24, 27, 30, 31

[Kotz et al., 2004] Kotz, D., Newport, C., Gray, R. S., Liu, J., Yuan, Y., and Elliott, C. (2004). Experimental evaluation of wireless simulation assumptions. In *Proceedings of the 7th ACM International Symposium on Modeling, Analysis and Simulation of Wireless and Mobile Systems*, pages 78–82. 25, 31, 40, 46, 76, 80

[Kurkowski et al., 2005] Kurkowski, S., Camp, T., and Colagrosso, M. (2005). MANET simulation studies: The incredibles. *ACM SIGMOBILE Mobile Computing and Communications Review*, 9(4):50–61. 21, 24, 31, 33, 35, 41

[Lantz et al., 2010] Lantz, B., Heller, B., and McKeown, N. (2010). A network in a laptop: rapid prototyping for software-defined networks. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, pages 1–6. ACM. 29, 30

[Levis et al., 2003] Levis, P., Lee, N., Welsh, M., and Culler, D. (2003). TOSSIM: Accurate and scalable simulation of entire TinyOS applications. In *Proceedings of the 1st International Conference on Embedded Networked Sensor Systems*, pages 126–137. 28

[Liu et al., 2024] Liu, Y., Xie, J., Xing, C., Xie, S., and Luo, X. (2024). Self-organization of UAV networks for maximizing minimum throughput of ground users. *IEEE Transactions on Vehicular Technology*, 73(8):11743–11755. 32

UNIVERSIDAD SERGIO ARBOLEDA

[Montresor and Jelasity, 2009] Montresor, A. and Jelasity, M. (2009). PeerSim: A scalable P2P simulator. In *Proceedings of the 9th IEEE International Conference on Peer-to-Peer Computing (P2P 2009)*, pages 99–100, Seattle, WA, EE.UU. IEEE. 23, 29

[Murray et al., 2010] Murray, D., Dixon, M., and Koziniec, T. (2010). An experimental comparison of routing protocols in multi hop ad hoc networks. In *2010 Australasian Telecommunication Networks and Applications Conference*, pages 159–164. 31

[Neumann et al., 2008] Neumann, A., Aichele, C., Lindner, M., and Wunderlich, S. (2008). Better approach to mobile ad-hoc networking (B.A.T.M.A.N.). IETF Internet-Draft, draft-openmesh-b-a-t-m-a-n-00. Trabajo en progreso. https://datatracker.ietf.org/doc/draft-openmesh-b-a-t-m-a-n/. 31, 41, 57

[Ni et al., 1999] Ni, S.-Y., Tseng, Y.-C., Chen, Y.-S., and Sheu, J.-P. (1999). The broadcast storm problem in a mobile ad hoc network. In *Proceedings of the 5th Annual ACM/IEEE International Conference on Mobile Computing and Networking*, pages 151–162. 32, 48, 71, 80

[Open Mesh, 2026] Open Mesh (2026). batman-adv: B.A.T.M.A.N. advanced, implementación en el núcleo de linux. https://www.open-mesh.org/projects/batman-adv/wiki. Consultado en septiembre de 2026. 25, 31, 57

[Österlind et al., 2006] Österlind, F., Dunkels, A., Eriksson, J., Finne, N., and Voigt, T. (2006). Cross-level sensor network simulation with COOJA. In *Proceedings of the 31st IEEE Conference on Local Computer Networks (LCN 2006)*, pages 641–648. IEEE. 28, 30

[Pawlikowski et al., 2002] Pawlikowski, K., Jeong, H.-D. J., and Lee, J.-S. R. (2002). On credibility of simulation studies of telecommunication networks. *IEEE Communications Magazine*, 40(1):132–139. 21, 24, 31, 35, 41

[Perkins et al., 2003] Perkins, C., Belding-Royer, E., and Das, S. (2003). Ad hoc on-demand distance vector (AODV) routing. RFC 3561, RFC Editor. 28

[Perkins and Bhagwat, 1994] Perkins, C. E. and Bhagwat, P. (1994). Highly dynamic destination-sequenced distance-vector routing (DSDV) for mobile computers. *ACM SIGCOMM Computer Communication Review*, 24(4):234–244. 28

[Poduri and Sukhatme, 2004] Poduri, S. and Sukhatme, G. S. (2004). Constrained coverage for mobile sensor networks. In *IEEE International Conference on Robotics and Automation, 2004*, pages 165–171. 24, 32, 47, 76

[Postel, 1980] Postel, J. (1980). User datagram protocol. RFC 768, RFC Editor. 47

[Postel, 1981] Postel, J. (1981). Internet control message protocol. RFC 792, RFC Editor. 47

UNIVERSIDAD
SERGIO ARBOLEDA

[Pygame Community, 2026] Pygame Community (2026). pygame-ce, versión 2.5. https://github.com/pygame-community/pygame-ce. Consultado en noviembre de 2026. 49

[Python Software Foundation, 2026] Python Software Foundation (2026). Python, versión 3.14. https://www.python.org/. Consultado en noviembre de 2026. 48, 84

[Reina et al., 2015] Reina, D. G., Askalani, M., Toral, S. L., Barrero, F., Asimakopoulou, E., and Bessis, N. (2015). A survey on multihop ad hoc networks for disaster response scenarios. *International Journal of Distributed Sensor Networks*, pages 1–16. 21, 24, 32

[Riley and Henderson, 2010] Riley, G. F. and Henderson, T. R. (2010). The ns-3 network simulator. In *Modeling and Tools for Network Simulation*, pages 15–34. Springer, Berlin, Heidelberg. 21, 23, 27

[Sánchez Cifuentes et al., 2026] Sánchez Cifuentes, J. F. et al. (2026). Simulador_batman: simulador de una red B.A.T.M.A.N. Repositorio de código, https://github.com/jofsanchezci/Simulador_BATMAN. Rama `develop`. Consultado en agosto de 2026. 21, 23, 35

[Seidel and Rappaport, 1992] Seidel, S. Y. and Rappaport, T. S. (1992). 914 MHz path loss prediction models for indoor wireless communications in multifloored buildings. *IEEE Transactions on Antennas and Propagation*, 40(2):207–217. 33, 46, 80

[Seither et al., 2011] Seither, D., König, A., and Hollick, M. (2011). Routing performance of wireless mesh networks: A practical evaluation of BATMAN advanced. In *2011 IEEE 36th Conference on Local Computer Networks*, pages 897–904. 23, 31, 43

[Stingl et al., 2011] Stingl, D., Gross, C., Rückert, J., Nobach, L., Kovacevic, A., and Steinmetz, R. (2011). PeerfactSim.KOM: A simulation framework for peer-to-peer systems. In *2011 International Conference on High Performance Computing and Simulation*, pages 577–584. 29

[Tazaki et al., 2013] Tazaki, H., Uarbani, F., Mancini, E., Lacage, M., Camara, D., and Turletti, T. (2013). Direct code execution: revisiting library OS architecture for reproducible network experiments. In *Proceedings of the Ninth ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, pages 217–228. ACM. 28, 30, 33, 77

[The ns-3 Project, 2026] The ns-3 Project (2026). ns-3 model library. https://www.nsnam.org/docs/models/html/routing-overview.html. Documentación oficial. Consultado en noviembre de 2026. 27, 28, 77

[Varga and Hornig, 2008] Varga, A. and Hornig, R. (2008). An overview of the OMNeT++ simulation environment. In *Proceedings of the 1st International Conference*

UNIVERSIDAD SERGIO ARBOLEDA

on Simulation Tools and Techniques for Communications, Networks and Systems (SIMUTools '08), Marseille, Francia. ICST. 21, 23, 27, 28

[Wehrle et al., 2010] Wehrle, K., Güneş, M., and Gross, J., editors (2010). *Modeling and Tools for Network Simulation*. Springer, Berlin, Heidelberg. 21, 27

[Zeng et al., 1998] Zeng, X., Bagrodia, R., and Gerla, M. (1998). GloMoSim: A library for parallel simulation of large-scale wireless networks. In *Proceedings of the 12th Workshop on Parallel and Distributed Simulation (PADS '98)*, pages 154–161, Banff, Alberta, Canadá. 27

[Zou and Chakrabarty, 2003] Zou, Y. and Chakrabarty, K. (2003). Sensor deployment and target localization based on virtual forces. In *IEEE INFOCOM 2003*, pages 1293–1303. 32, 47

UNIVERSIDAD
SERGIO ARBOLEDA